Department of Computer Science and Engineering
University of Moratuwa

Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing

Software Architecture Document

Version <2.0>

Project ID(PID)	11
Group Number	25
Mentor	Mr. Bhanuka Siriwardhana

Group Members

Registration No.	Name
230667F	Virusan T.
230670H	Vishula J.
230674A	R.P.M.Vithanage

 ©AutoVault LK, 2026

Revision History

Date	Version	Description	Author
15/07/2026	0.1	Project commenced; initial architecture scope and view model outline	Group 25
27/07/2026	0.5	Draft SAD : ADRs, use-case/logical/process views, API and security sections	Group 25
09/08/2026	1.0	Initial documentation release : figures inserted, section 1.3 glossary completed, document review	Group 25
10/08/2026	2.0	Corrected SAD/SRS sections that contradicted the reduced-team decision to keep the synchronous ETL→Marketplace write boundary (rejecting an event-driven refactor). Fixed: ADR-002, section 3.1 data isolation bullet, section 3.4.3 IAM table, section 9 Data View, section 11.1 Reliability, section 3.10 (SRS) , all previously stated "zero exceptions" / event-based writes, now correctly document the single scoped MarketplaceVehiclesWriteAdapter exception. Removed stray event-driven language from SRS FR-14, FR-18.1, FR-43. Updated team size/timeline (3→2 developers, 6 weeks remaining) in SAD section 3.2 and SRS section2.4.	Group 25

Individual Contributions

Index No.	Name	Contribution
230667F	Virusan T.	• Section 1.2 Scope • Section 3.1 Architectural goals • Section 3.3 ADR-004 - Rules-first normalization + Groq fallback • Section 3.4.5 Security - External dependency security (Groq) • Section 3.5.1 API architecture - Public API Gateway • Section 3.5.4 API architecture - Error response conventions • Section 3.6.2 Error handling - Search resilience • Section 3.8.1 Testing architecture - Testing layers • Section 3.8.2 Architecturally significant test cases • Section 4.1.3 Dealer listing management • Section 4.1.4 Hybrid natural language search • Section 5.1 Logical view - Overview • Section 5.2.2 Marketplace module packages • Section 5.2.x Shared normalize-embed library • Section 6.2–6.3 Guest / Buyer browse & search narratives • Section 8.2 Component layering

		• Section 9 Entity descriptions • Section 10 Size & performance view • Section 11 NFR traceability • Appendix A - Public API route catalogue • Figure 2 - Guest use-case diagram • Figure 3 - Buyer use-case diagram • Figure 8 - API Gateway route map • Logical package / component layering diagram
230670H	Vishula J.	• Section 1.3 Definitions & glossary • Section 3.2 Constraints • Section 3.3 ADR-002 - ETL Load cross-schema write • Section 3.3 ADR-003 - Step Functions orchestration • Section 3.4.3 Security - IAM least-privilege table • Section 3.5.3 API architecture - Asynchronous interfaces • Section 3.6.1 Error handling - ETL pipeline resilience • Section 3.6.3 Error handling - Notification resilience • Section 3.7.1 CI/CD - Repository structure • Section 3.7.2 CI/CD - GitHub Actions pipeline • Section 3.7.3 CI/CD - Environments • Section 3.8.3 Testing architecture - Test data / fixtures • Section 4.1.2 Bulk upload intake • Section 5.2.3 Ingestion & ETL module packages • Section 5.2.5 Notification module packages • Section 6.1 Process organisation & communication table • Section 6.6 ETL Map state / chunk processing sequence • Section 7 AWS topology & network partitioning • Section 7 S3 bucket layout & RDS Proxy • Section 8.1 Repository layout & migrations • Section 11 Reliability, security, performance summaries • Figure 1 - High-level architecture diagram • Figure 9a/9b - ETL error handling flowcharts • Figure 10 - CI/CD pipeline diagram • Figure 11/20 - Deployment diagram

230674A	Vithanage R.P.M.	• Section 1.1 Purpose • Section 1.4 References • Section 1.5 Document overview • Section 3.1 Modular monolith clarification paragraph (v1) • Section 3.3 ADR-001 - Shared RDS / schema-per-module • Section 3.3 ADR-005 - Internal API Gateway (east-west HTTP) • Section 3.3 ADR-006 - Tiered dealer onboarding • Section 3.3 ADR-007 - Modular monolith naming decision (v1) • Section 3.4.1 Security - Authentication model • Section 3.4.2 Security - RBAC / authorization • Section 3.4.4 Security - Data security • Section 3.5.2 API architecture - Internal / east-west routes • Section 4.1.1 Register & tiered authorization • Section 4.1.5 Monitor platform activity & audit logs • Section 5.2.1 Auth & User module packages • Section 5.2.4 Admin module packages • Section 6.4 Dealer registration & authorization sequence • Section 9 Schema-per-module isolation model • Section 9 Cross-schema grants • Appendix B - Admin routes & internal Auth calls • Revision history & document integration • Figure 4 - Dealer use-case diagram • Figure 5 - Administrator use-case diagram • Figure 6a/6b - Authentication sequence diagrams • Figure 7 - RBAC permission matrix

Table of Contents

Group Members 1

1. Introduction 9

1.1 Purpose 9

1.2 Scope 9

1.3 Definitions, Acronyms, and Abbreviations 9

1.4 References 14

1.5 Overview 14

2. Architectural Representation 15

3. Architectural Goals and Constraints 15

3.1 Architectural Goals 15

3.2 Constraints 16

3.3 Architecture Decision Records 17

3.3.1 ADR-001: Shared RDS Instance with Schema-Per-Service Isolation 17

3.3.2 ADR-002: Schema Boundaries for ETL Persistence 18

3.3.3 ADR-003: Step Functions Orchestration over Monolithic ETL Lambda 19

3.3.4 ADR-004: Rules-First Normalization with Groq LLM Fallback 20

3.3.5 ADR-005: Internal API Gateway for East-West Service Communication 21

3.3.6 ADR-006: Tiered Dealer Onboarding (Individual vs. Business) 21

3.4 Security Architecture 22

3.4.1 Authentication Model 22

Figure 03: Authentication sequence - protected API call. The browser sends a request with an access token through the API Gateway to the target service. The service verifies the token locally; if valid, it fetches data and returns the resource. If invalid, the client receives an access-denied response. The Auth Service is not called on every request. 25

3.4.2 Authorization Model (RBAC) 25

*bulk upload requires an approved business-tier account 26

3.4.3 Infrastructure Security (IAM) 26

3.4.4 Data Security 27

3.4.5 External Dependency Security 27
3.5 API and Interface Architecture 27
3.5.1 Public API Gateway (North-South) 27
3.5.2 Internal API Gateway (East-West) 28
3.5.3 Asynchronous Interfaces 29
3.6 Error Handling and Resilience 29
3.6.1 ETL Pipeline Resilience 29
3.6.2 Search Resilience 30
3.6.3 Notification Resilience 30
3.7 CI/CD and Environment Architecture 32
3.7.1 Repository Structure 32
3.7.2 CI/CD Pipeline (GitHub Actions) 33
3.7.3 Environments 33
3.8 Testing Architecture 34
3.8.1 Testing Layers 34
3.8.2 Architecturally Significant Test Cases 34
3.8.3 Test Data 35
4. Use-Case View 35
Guest Use Cases 35
Buyer Use Cases 36
Dealer Use Cases 37
Administrator Use Cases 39
4.1 Use-Case Realizations 40
4.1.3 Process and Review Pending Listings 42
4.1.5 Monitor Platform Activity and Audit Logs 47
5. Logical View 48
5.1 Overview 48
5.2 Architecturally Significant Design Packages 48
6. Process View 53
6.1 Process Organisation and Communication 53
6.2 Bulk Inventory Upload & ETL Orchestration Activity 53
6.3 Hybrid Natural-Language Search Activity 55
6.4 Dealer Registration & Tiered Authorization Sequence 56
6.5 Bulk Upload Intake & SQS Decoupling Sequence 56
6.6 ETL Chunk Processing (The Map State) Sequence 57
6.7 Intelligent Search Execution Sequence 58
6.8 Dealer Account Verification State 59
6.9 Vehicle Listing State (Row-Level Lifecycle) 61
7. Deployment View 63
8. Implementation View 64
8.1 Overview 64
8.2 Layers 65
9. Data View (optional) 66
10. Size and Performance 67

11. Quality **69**
11.7 Requirements Traceability Summary 70
Appendix A: Requirements Traceability Matrix (Full) 71
Appendix B: Public API Route Catalogue (Illustrative) 72
Marketplace Service 72
Ingestion & ETL Service 73
Admin Service 73
Appendix C: RBAC Permission Matrix 74

Software Architecture Document

1. Introduction

The paper describes the full architectural design of Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing System using various views within architecture. The paper captures the important architectural decisions that were made with respect to the system following the 4+1 Architecture Model (Use-Case View, Logical View, Process View, Deployment View, Implementation View, and a further Data View because of the amount of persistent data in the system)..

1.1 Purpose

The purpose of this document Software Architecture Document (SAD) is to make sure that the developers/testers know about the design/architecture and the reasoning behind it. This document will cover the architectural decisions that have been made, service boundaries, orchestration approach, data ownership, and deployment strategy so that the actual implementation is consistent with this architecture instead of doing it differently.

1.2 Scope

The system enables vehicle dealers to upload their inventories in bulk (CSV/JSON along with a ZIP file of images), with the input automatically processed through an asynchronous pipeline that validates, normalizes, and enriches the vehicle data before making it available to buyers through both filter-based and natural-language search. This document details the architecture of all five backend modules(Auth & User, Marketplace, Ingestion & ETL, Admin, Notification), the React-based frontend application, and the AWS cloud infrastructure connecting them.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
Access token	Short-lived JWT issued at login; sent with API requests to authorize access to protected resources.
Active	Canonical listing status for a live, publicly searchable vehicle listing (replaces informal “LIVE” in earlier drafts).
ADR	Architecture Decision Record, a documented significant design choice with context, decision, consequences, and alternatives.
ALB	Application Load Balancer, optional AWS load balancer used for internal (east-west) HTTP routing between services.
API Gateway	AWS managed HTTP entry point; the public gateway handles browser traffic (north-south); a private internal gateway handles service-to-service calls (east-west).

AWS	Amazon Web Services, cloud platform hosting all production components except the external Groq LLM API.
Chunk	A group of 100 inventory rows processed together as one Step Functions Map iteration.
Choice state	A Step Functions construct that branches execution based on a condition, with no Lambda invocation or cost.
Cold start	The added latency incurred when AWS Lambda must initialize a new execution environment.
Confidence score	Measure of parse/normalization certainty. In ETL, the minimum required-field score for a row; in search, based on token coverage and numeric signals. Rows/queries below 0.6 may invoke Groq fallback.
DLQ	Dead-Letter Queue, Amazon SQS queue where messages are routed after repeated processing failures.
DTO	Data Transfer Object, validated request/response shape used at NestJS controller boundaries.
East-west traffic	Service-to-service HTTP calls within the AWS VPC (e.g., Admin Service -> Auth Service via internal API Gateway).
ECR	Amazon Elastic Container Registry, stores Docker images deployed to AWS Lambda.
ETL	Extract, Transform, Load , the automated pipeline that turns a dealer's raw CSV/JSON/ZIP upload into validated, searchable listings.
FR / NFR	Functional Requirement / Non-Functional Requirement, identifiers from the SRS traceable to SAD elements.
Groq	External LLM inference provider used as a fallback normalizer for low-confidence ETL rows and unresolved natural-language search tokens, never the primary parser.
Human-in-the-loop	Design gate requiring dealer review before AI-inferred listing data is published; ETL-loaded listings enter pending_review until confirmed.
IaC	Infrastructure as Code : AWS resources defined and provisioned via Terraform (cloud-infrastructure repository).

IAM	AWS Identity and Access Management : least-privilege execution roles scoped per Lambda function (e.g., S3 prefix access, RDS schema grants, SQS publish/consume).
JWT	JSON Web Token - stateless bearer token encoding user identity and role claims; validated locally by each service using a shared public key (NFR-13).
Lambda	AWS Lambda : serverless compute running NestJS HTTP handlers and the eleven ETL stage functions.
LLM	Large Language Model : in this system, Groq's hosted model used strictly as a fallback normalizer/repairer, not as the primary parser.
Map state	Step Functions construct that fans out and processes an array of chunk descriptors in parallel (MaxConcurrency: 10).
MiniLM	The all-MiniLM-L6-v2 sentence-embedding model used to generate 384-dimensional vectors for semantic search and listing embeddings.
Model-parity constraint	Non-negotiable requirement that ingestion and search use the same normalization dictionaries and embedding model/library version (FR-22.1, NFR-26.1).
Human-in-the-loop	Design gate requiring dealer review before AI-inferred listing data is published; ETL-loaded listings enter pending_review until confirmed.
IaC	Infrastructure as Code : AWS resources defined and provisioned via Terraform (cloud-infrastructure repository).
IAM	AWS Identity and Access Management : least-privilege execution roles scoped per Lambda function (e.g., S3 prefix access, RDS schema grants, SQS publish/consume).
JWT	JSON Web Token : stateless bearer token encoding user identity and role claims; validated locally by each service using a shared public key (NFR-13).
Lambda	AWS Lambda : serverless compute running NestJS HTTP handlers and the eleven ETL stage functions.
LLM	Large Language Model : in this system, Groq's hosted model used strictly as a fallback normalizer/repairer, not as the primary parser.
Map state	Step Functions construct that fans out and processes an array of chunk descriptors in parallel (MaxConcurrency: 10).

MarketplaceVehiclesWriteAdapter	Sole application class permitted to exercise the ADR-002 cross-schema write grant into marketplace.vehicles and marketplace.vehicle_images.
MiniLM	The all-MiniLM-L6-v2 sentence-embedding model used to generate 384-dimensional vectors for semantic search and listing embeddings.
Model-parity constraint	Non-negotiable requirement that ingestion and search use the same normalization dictionaries and embedding model/library version (FR-22.1, NFR-26.1).
NestJS	TypeScript backend framework used for all modules; enforces layered structure (controllers, services, repositories, adapters).
NIC	National Identity Card : verification document required for Individual-tier dealer registration.
North-south traffic	Client browser to backend traffic via the public API Gateway over HTTPS.
Pending review	Listing status after ETL Load; not publicly searchable until the dealer explicitly confirms AI-inferred data (FR-42).
pg_trgm	PostgreSQL extension enabling trigram-based fuzzy text matching (used for make/model search and ETL normalization).
pgvector	PostgreSQL extension enabling storage and similarity search over vector embeddings (vector(384) on listings).
Provenance	Metadata recording whether a field's value came from deterministic rules or LLM inference, and with what confidence.
RBAC	Role-Based Access Control : authorization enforced via JWT role claims and NestJS RolesGuard (Guest, Buyer, Dealer, Administrator).
RDS	Amazon Relational Database Service : hosts the shared PostgreSQL instance with schema-per-service isolation (ADR-001).
RDS Proxy	Connection pooler between Lambda functions and RDS; required to bound database connections under Step Functions Map concurrency (NFR-10).
Refresh token	Long-lived opaque token used to obtain new access tokens without re-login; stored hashed server-side and revocable on logout (FR-05).
Rejected records	Table (ingestion.rejected_records) storing rows that fail validation after rules and AI normalization, with actionable error reasons.

Rules-first normalization	Deterministic dictionary/regex/pg_trgm parsing applied before any LLM call; Groq invoked only when confidence falls below 0.6 (ADR-004).
S3	Amazon Simple Storage Service : stores immutable raw uploads (raw/), staging JSON between ETL states (staging/), dealer verification documents, and processed vehicle images.
Schema-per-service	Logical database isolation model: one RDS instance, five service-owned schemas (auth, marketplace, ingestion, notification, admin), each with a dedicated PostgreSQL role.
SES	Amazon Simple Email Service : sends transactional email notifications (upload complete/fail, dealer approval, etc.).
SPA	Single Page Application : React web frontend served from S3/CloudFront.
SQS	Amazon Simple Queue Service : durably buffers upload job messages between the Ingest API and Step Functions.
SSM	AWS Systems Manager Parameter Store : stores configuration and secrets (e.g., JWT keys, Groq API key) alongside Secrets Manager.
State machine	An AWS Step Functions execution orchestrating a sequence of Lambda invocations and branching logic for one upload job.
Step Functions	AWS orchestration service coordinating the eleven ETL Lambda stages per upload with declarative Retry/Catch (ADR-003).
Terraform	Infrastructure-as-code tool used in cloud-infrastructure for reproducible AWS provisioning and deployment.
vehicle_dictionaries	Marketplace-owned reference table (makes, models, fuel, transmission, aliases) backing pg_trgm fuzzy matching; read by Ingestion via scoped grant and in-memory cache.
VPC	Virtual Private Cloud : network boundary enclosing Lambda functions, RDS, and internal API Gateway.
ADR	Architecture Decision Record - a documented significant design choice with context and consequences.
East-west traffic	Service-to-service HTTP calls within the AWS VPC (e.g., Admin -> Auth).
North-south traffic	Client browser to backend via public API Gateway.

Map state	Step Functions construct that processes an array of chunk descriptors in parallel (MaxConcurrency: 10).
Confidence score	Ratio of consumed meaningful tokens to unparsed tokens (search) or minimum required-field score (ETL). Threshold: 0.6.
pending_review	Listing status after ETL Load; not publicly searchable until dealer confirms (FR-42).
active	Canonical term for a live, publicly searchable listing (replaces informal "LIVE" in earlier SAD drafts).
MarketplaceVehicles WriteAdapter	Sole class permitted to exercise ADR-002 cross-schema write grant.
vehicle_dictionaries	Marketplace-owned reference table for pg_trgm fuzzy matching; Ingestion reads via scoped grant + in-memory cache.

1.4 References

- [1] Group 25 (PID 11), Project Proposal - Second-Hand Vehicle Marketplace, University of Moratuwa, 2026.
- [2] Group 25 (PID 11), Feasibility Study - Second-Hand Vehicle Marketplace, University of Moratuwa, 2026.
- [3] Group 25 (PID 11), Software Requirements Specification v2.0 - Second-Hand Vehicle Marketplace, University of Moratuwa, 2026.
- [4] Group 25 (PID 11), ETL Pipeline - Design: Dealer vehicle-upload ingestion (Step Functions, rules-first normalization, LLM fallback), 31 July 2026.
- [5] Amazon Web Services, "AWS Step Functions Documentation." Available: https://docs.aws.amazon.com/step-functions/
- [6] Amazon Web Services, "AWS Documentation." Available: https://docs.aws.amazon.com/
- [7] pgvector, "Vector Similarity Search for PostgreSQL." Available: https://github.com/pgvector/pgvector
- [8] Hugging Face, "Sentence-Transformers: all-MiniLM-L6-v2." Available: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2
- [9] NestJS, "NestJS Documentation." Available: https://docs.nestjs.com/
- [10] HashiCorp, "Terraform Documentation." Available: https://developer.hashicorp.com/terraform/docs

Diagrams in this document are drawn using Mermaid and draw.io; the source files are maintained alongside this document and referenced by filename in each figure caption once inserted.

1.5 Overview

The architectural perspectives this document uses, and the motivation behind them, are discussed in Section 2. The goals and constraints that shaped the architecture are described in Section 3. The

Use-Case View is presented in Section 4. Section 5 covers the Logical View (package- and class-level decomposition). The Process View (runtime behavior and concurrency) is described in Section 6. Section 7 covers the Deployment View (physical AWS topology). The Implementation View (source-level layering across repositories) is covered in Section 8. Section 9 covers the Data View. Size/Performance and Quality Attributes are discussed in Sections 10 and 11 respectively.

2. Architectural Representation

This document uses six views to describe the stakeholder concerns:

- **Use-Case View**: the functional behavior visible to Guests, Buyers, Dealers, and Administrators; ties the other views to concrete scenarios.
- **Logical View**: decomposition of the system into modular packages, domain modules, and architecturally significant classes.
- **Process View**: the system's runtime behavior concurrent execution, sequencing, and process communication (HTTP, SQS, Step Functions state transitions, polling).
- **Deployment View**: the physical AWS platform the software runs on, and the mapping of logical components onto it.
- **Implementation View**: source-code organization repositories, layers within each service, and layering rules.
- **Data View**: the persistent data model the shared PostgreSQL schema and the S3 object layout.

Together these views implement the 4+1 architectural view model, extended with an explicit Data View given the system's substantial persistent-data and search-indexing requirements.

3. Architectural Goals and Constraints

3.1 Architectural Goals

- **Serverless first:** all backends are hosted on AWS Lambda as part of the AWS Free Tier plan and to manage idle server capacity in a system that is highly bursty and driven by uploads.
- **Orchestrated ETL via per-stage Lambda :** Ingestion and ETL pipelines have been implemented as AWS Step Functions state machines with a dedicated per-stage Lambda each, replacing the previous implementation where one single ETLWorker Lambda was being used. This brings in declarative retry mechanism, memory sizing per stage, and live execution graph, but with a trade-off of having more Lambdas to manage.
- **Rules-first + LLM fallback normalization:** Deterministic dictionary / regex based normalization takes care of the vast majority of fields from dealers for free; Groq (LLM) is only called when the rules are not able to determine the values.
- **Human-in-the-loop gate:** no value LIVE by Groq is released without a review by the dealer.Uploaded listings are placed in a PENDING_REVIEW state and promoted to LIVE only via dealer's explicit action.Model-parity constraint (must hold): the embedding model and normalization dictionaries employed at ingestion and query time have to be exactly the

same, from the same library, otherwise the two halves of the system are effectively using different vocabularies and vector spaces.

- **Two-tier dealer onboarding process:** rather than a single path for all dealers, the system distinguishes an Individual tier (just NIC, manual listing creation, a limited number of live listings) and a Business tier (certificate of business registration, unlimited bulk CSV/JSON/ZIP upload).Verification constraints and platform capabilities scale by tiers together so a casual individual seller will not be asked for paperwork that would make sense only for a small dealership and bulk uploads won't be enabled without corresponding business documentation.

- **Cross-service data isolation model:** within the shared RDS instance, each service exclusively owns its schema. Reads may cross service schemas via scoped, service-specific database roles (e.g. admin_service_role may SELECT from marketplace and ingestion tables for dashboarding). Writes stay owned per service, with one narrow, database-enforced exception: the Ingestion & ETL pipeline's Load stage writes into marketplace.vehicles under a column-scoped grant, confined in code to MarketplaceVehiclesWriteAdapter and made retry-safe via an idempotent upsert. This exception was deliberately scoped rather than generalized , it is not a precedent for services writing freely into each other's schemas.

- **East-west service communication:** north-south requests from browser to services are passed through public API Gateway, while east-west calls between services (for example, Admin approving/deleting dealer), which are made using regular HTTP without Lambda, pass through internal API Gateway, thus providing identical code execution locally with Docker Compose and on the cloud platform AWS.

- **Shared reference data for fuzzy matching:** The pg_trgm dictionaries (including those with alias promotion) exist in one tiny reference table called vehicle_dictionaries, owned by marketplace-service. The ingestion-service queries it, caches a snapshot of it into memory upon container startup, using the same mechanism that is used by our ETL pipeline for Groq's make/model lookup, as opposed to embedding dictionaries into a shared library (not capable of supporting the indexed similarity() call) or an HTTP call per token.

- **One shared PostgreSQL instance, service-owned schemas:** The team made a conscious decision to use one Amazon RDS with per-service schema ownership instead of isolated per-service databases, considering the team's size and academic time frame; this ensures that issues such as migration history remain in one spot but the application code honors service boundaries.

- **Security by default**: JWT-based stateless authentication with revocable refresh tokens, role-based access control, and least-privilege IAM permissions scoped per Lambda function.

3.2 Constraints

- **Team and timeline constraints:** originally scoped for three undergraduate developers across a fixed academic schedule (26 June – 3 October); as of 2026-08-10, delivered by three developers with six weeks remaining, which further favours a small number of well-justified architectural decisions over a maximally "correct" but harder-to-execute design including declining refactors, like an event-driven ETL→Marketplace boundary, that don't solve an active problem.

- **Groq as the one non-AWS dependency**: every other component in the system lives inside AWS, but Groq is an external third-party API outside that boundary.Meaning it's the single point where the architecture must assume unavailability (rate limits, timeouts, outages) rather than treat failure as exceptional.

- **Step Functions' 256 KB state payload limit**: a hard AWS platform ceiling, not a design choice. It's what forces the "pass S3 keys between states, never row data" pattern used throughout the pipeline.

- **Zero-cost/open-source tooling constraint**: every technology choice (NestJS, React, PostgreSQL, pgvector, Terraform, GitHub Actions) was made partly *because* it's free.

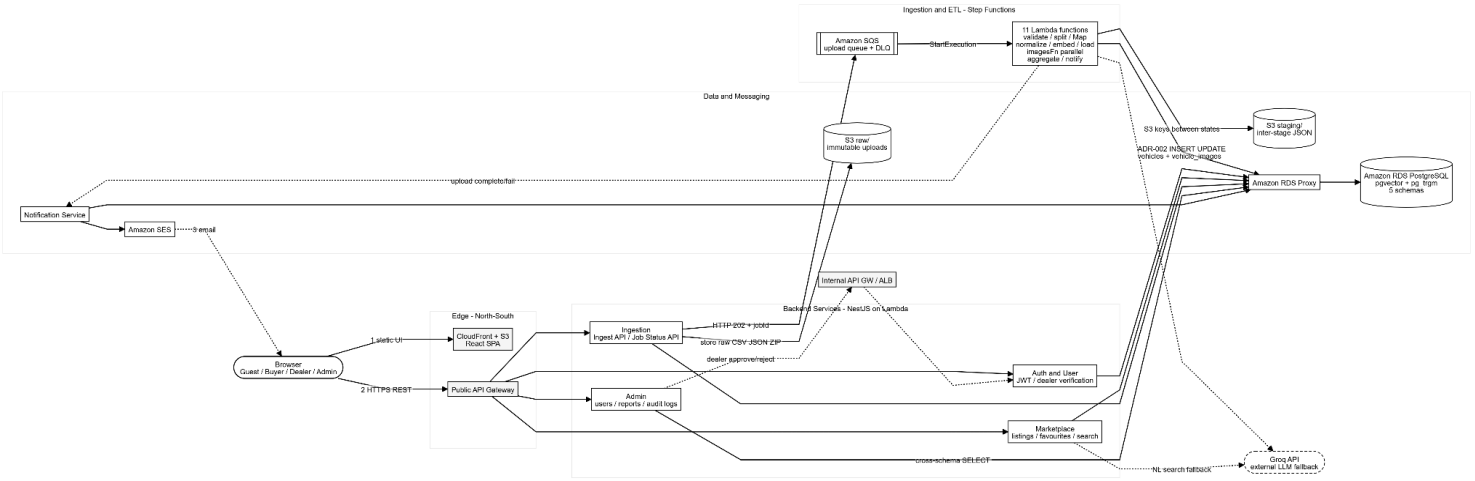


Figure 1. High-level architecture overview for AutoVault LK. The React SPA, public API Gateway, five NestJS modules on Lambda, the Step Functions ETL pipeline, and shared AWS infrastructure (RDS, S3, SQS, SES). Solid arrows indicate synchronous calls or direct writes; dotted arrows indicate asynchronous or event-driven flows.

3.3 Architecture Decision Records

3.3.1 ADR-001: Shared RDS Instance with Schema-Per-Service Isolation

Status: Accepted
Date: July 2026
Deciders: Group 25

Context
The architecture is made up of five backend modules, all of which have their own individual data domain. The team that is involved includes three undergraduates who are working in the academic time frame (June 26 – October 3) and using AWS free tier budget. If we go for fully isolated databases for each service, costs will increase.

Decision

Use an Amazon RDS PostgreSQL instance (pgvector and pg_trgm extensions) and logically isolate by creating five schemas for each of the services as auth, marketplace, ingestion, notifications, and admin. Each service will use a specific PostgreSQL user that has the least privilege on its schema only. Any cross-schema access must be allowed only via clearly defined grants.

Consequences

- **Positive:** Single migration history; simpler local development; foreign keys (e.g., marketplace.vehicles.dealer_id -> auth.users.id) remain enforceable at the database level; Free Tier budget is preserved.
- **Negative:** Services share a physical failure domain; schema migration coordination is required; true independent scaling of data stores is not possible.
- **Mitigation:** Service-owned schemas and role-based grants prevent accidental cross-service writes; centralized migrations are owned by a single database/ folder with explicit deployment ordering documented in the Implementation View.

Alternatives Considered

Alternative	Reason rejected
Five separate RDS instances	Cost, ops burden, and FK complexity outweigh benefits at prototype scale
Single monolithic schema	No logical service boundaries; violates microservice ownership model
Shared schema, no roles	Cannot enforce isolation; unacceptable for a service-oriented modular architecture

3.3.2 ADR-002: Schema Boundaries for ETL Persistence

Status: Accepted
Date: July 2026
Deciders: Group 25

Context
The Ingestion & ETL pipeline processes bulk dealer uploads through AWS Step Functions with a Map state (max concurrency: 10, 100 rows per chunk). The final Load stage must persist validated listings into the Marketplace domain. This service-oriented modular architecture defaults to write isolation; each service writes only within its own schema to prevent cross-schema contamination and preserve clear domain ownership. This default holds everywhere except one boundary, addressed in the Decision below.

Decision
Write isolation is the default across all five schemas: each service writes only within its own schema, enforced via least-privilege database roles. The Ingestion & ETL pipeline's Load stage is the sole exception. Because Ingestion and Marketplace have a strict 1:1 producer–consumer relationship for listing data one Map-state pipeline, one destination table, no independent scaling need on either side the Load stage (loadFn) writes validated listings directly into marketplace.vehicles under a narrow, column-scoped grant, confined in code to a single adapter class, MarketplaceVehiclesWriteAdapter. This was evaluated against introducing an event-driven boundary (SQS + consumer + DLQ + idempotency-key handling) and deliberately rejected: a queue would add failure surface ordering, consumer lag, duplicate delivery, DLQ monitoring without solving a problem that exists. All other stage writes (upload_jobs, etl_stage_logs) remain within the ingestion schema as before.

Consequences

- Positive: avoids an unnecessary event-driven layer for a boundary with no fan-out; write path stays narrow (two verbs, one table pair), reviewable, and enforced at the database level rather than by convention alone; reliability is handled once, at the correct layer (Step Functions Retry/Catch + idempotent upsert), not duplicated by a second retry/DLQ mechanism.
- Negative: this is a real, if narrow, exception to schema ownership it must stay confined to the one adapter class and not be treated as precedent for other cross-schema writes.
- Mitigation: idempotent Load logic (FR-41, ON CONFLICT on (upload_job_id, registration_number) or equivalent) makes retries safe; ETL-loaded listings enter pending_review before becoming visible (FR-42); the grant is column-scoped, not table-wide.

Alternatives Considered

Alternative	Reason rejected
Event-driven boundary (SQS + consumer + DLQ + idempotency keys)	1:1 producer–consumer with no independent scaling need; adds failure surface (ordering, lag, duplicate delivery, DLQ monitoring) without solving a real problem; Step Functions already provides retry/dead-lettering at the pipeline level
Synchronous HTTP API per row/chunk during Load	Adds network latency and failure modes under high Map concurrency
Fully generalized cross-schema write access (no adapter confinement)	Would erase the isolation model for the other four service boundaries; not justified by a single 1:1 relationship

3.3.3 ADR-003: Step Functions Orchestration over Monolithic ETL Lambda

Status: Accepted
Date: July 2026
Deciders: Group 25

Context
An earlier design used a single SQS-triggered ETLWorker Lambda to process entire uploads. This approach hit AWS Lambda's 15-minute timeout, could not size memory per task (embedding vs. image processing vs. rule-based parsing), and required hand-written retry and completion tracking.

Decision
Replace the monolithic worker with an AWS Step Functions state machine orchestrating eleven purpose-specific Lambda functions per upload job. Stages pass S3 object keys between states (never inline row data) to respect Step Functions' 256 KB state payload limit. The state machine uses a Map state for parallel chunk processing (MaxConcurrency: 10) and a Parallel state for concurrent row-processing and image-processing branches.

Consequences

- **Positive:** Declarative Retry/Catch per stage; independent memory sizing (e.g., embed function at 3008 MB, images function at 2048 MB); live execution graph in AWS Console; a failed Groq step degrades one chunk rather than killing the entire upload.

- **Negative:** More Lambda functions to maintain (~61 invocations per 1,000-row upload vs. ~10 for a fused worker); higher total invocation count.
- **Mitigation:** Shared NestJS module structure across ETL handlers; etl_stage_logs table records per-stage status for observability.

Alternatives Considered

Alternative	Reason rejected
Single monolithic ETL Lambda	Timeout and memory constraints; poor retry granularity
ECS/Fargate long-running worker	Outside Free Tier budget; ops overhead
SQS-only fan-out without Step Functions	No declarative orchestration; manual completion tracking

3.3.4 ADR-004: Rules-First Normalization with Groq LLM Fallback

Status: Accepted
Date: July 2026
Deciders: Group 25

Context
Dealer-uploaded inventory data is frequently messy: typos in make/model, abbreviated prices ("8.5m"), inconsistent fuel type spellings. A pure LLM approach would be costly, rate-limit sensitive, and prone to hallucinated values. A pure rules approach fails on genuinely ambiguous rows.

Decision
Apply a deterministic rule-based parser first (dictionary lookups, regex, pg_trgm fuzzy matching). Calculate a per-field confidence score and a row-level confidence score (minimum of required field scores, never an average). Rows scoring below 0.6 are routed to a dedicated Groq Lambda for LLM-based normalization of unresolved fields only. All LLM outputs pass a whitelist validation layer before merge. If Groq is unavailable, the pipeline proceeds with rules-only results (FR-42.1).

The same rules-first structure applies to natural-language search parsing in the Marketplace Service (FR-21), with a shared normalize library ensuring vocabulary parity between ingestion and search (FR-22.1, NFR-26.1).

Consequences

- **Positive:** Majority of rows processed at zero LLM cost; blast radius of AI errors is bounded to low-confidence fields; graceful degradation on Groq outage.
- **Negative:** Groq is the single non-AWS dependency; free-tier rate limits require Retry/Catch and fallback logic.
- **Mitigation:** Recurring Groq corrections are promoted to vehicle_dictionaries.aliases (FR-21.4), reducing future LLM calls.

Alternatives Considered

Alternative	Reason rejected

LLM-first for all rows	Cost, rate limits, hallucination risk
Rules-only, no LLM	Fails on ambiguous dealer data (see SRS Appendix B sample row 2)
Per-field HTTP call to Marketplace dictionary API during ETL	Latency under Map concurrency

3.3.5 ADR-005: Internal API Gateway for East-West Service Communication

Status: Accepted
Date: July 2026
Deciders: Group 25

Context
Some operations require one service to invoke another (e.g., Admin approving a dealer must update auth.dealer_profiles). Direct Lambda-to-Lambda invocation works in AWS but diverges from the local Docker Compose development environment, where services communicate over HTTP.

Decision

All service-to-service (east-west) calls will be done via an internal API Gateway or Application Load Balancer using regular HTTP calls. There is consistency in the URL pattern and structure of the request whether you are in local development mode or on AWS. Admin Service calls Auth Service for approve/reject of the dealer; Auth Service does the write in their schema. Admin Service writes to their own admin.audit_logs table.

North-south traffic (browser to backend) continues through the public API Gateway.

Consequences

- **Positive:** Identical code paths in local and cloud environments; no direct cross-schema writes for admin actions; consistent HTTP semantics and error handling.
- **Negative:** Additional network hop and infrastructure component; internal gateway must be secured (VPC, no public exposure).
- **Mitigation:** Internal gateway is private; JWT validation on all internal routes; least-privilege IAM per Lambda.

Alternatives Considered

Alternative	Reason rejected
Direct Lambda invocation	Different code path in Docker Compose vs. AWS
Admin writes directly to auth schema	Second cross-schema write exception; violates isolation
Shared database, no service APIs	Removes service boundaries entirely

3.3.6 ADR-006: Tiered Dealer Onboarding (Individual vs. Business)

Status: Accepted
Date: July 2026
Deciders: Group 25

Context
The platform serves both casual individual sellers and registered dealerships. Requiring business registration documents from every seller would exclude legitimate individual sellers. Granting bulk CSV/ZIP upload to unverified individuals would invite abuse and data quality problems.

Decision
Dealers declare a type at registration: **Individual** or **Business**. Verification requirements and platform capabilities scale with tier:

Tier	Required documents	Capabilities after approval
Individual	NIC (National Identity Card)	Manual listing creation; capped active listings (configurable limit)
Business	Business registration certificate (+ optional signatory NIC)	Uncapped bulk CSV/JSON/ZIP upload via Ingest API

All dealer registrations enter verification_status = pending until an Administrator approves (FR-02). Bulk upload intake (Use Case 4.1.2) requires an authenticated JWT and an approved **Business-tier** profile.

Consequences

- **Positive:** proportionate verification burden; bulk upload gated behind business documentation; aligns with real marketplace trust models.
- **Negative:** SRS FR-26 currently states any Dealer may upload - **SRS must be updated** to reflect Business-tier restriction, or FR-26 must be annotated with tier precondition.
- **Mitigation:** Ingest API validates dealer tier from JWT claims or a scoped read of auth.dealer_profiles before accepting upload.

Alternatives Considered

Alternative	Reason rejected
Single verification path for all dealers	Over-burdens individuals or under-protects bulk upload
No tier distinction	Cannot gate bulk upload meaningfully

3.4 Security Architecture

This subsection expands the security considerations referenced in Section 11 (Quality) and SRS Section 3.4 (NFR-13 through NFR-20). It describes how authentication, authorization, secrets, and infrastructure permissions are applied across the deployment topology.

3.4.1 Authentication Model

The system uses **stateless JWT-based authentication** with revocable refresh tokens (FR-03, FR-04, FR-05, NFR-13).

Token	Purpose	Storage	Lifetime
Access token	API authorization	Client memory (not localStorage for XSS mitigation where possible)	Short-lived (e.g., 15 minutes)
Refresh token	Obtain new access token without re-login	HttpOnly cookie or secure client store	Longer-lived (e.g., 7 days)

Login flow:

1. Client submits email/password to Auth Service via public API Gateway.
2. Auth Service verifies salted password hash (FR-07).
3. On success, issues JWT access token (with role claim: buyer, dealer, admin) and refresh token.
4. Refresh token hash stored in auth.refresh_tokens; raw token never persisted (FR-05).

Token validation:
All HTTP-facing Lambdas validate JWT locally against a shared public key distributed via environment/SSM, avoiding a per-request network call to Auth Service (except for explicit token introspection if added later).

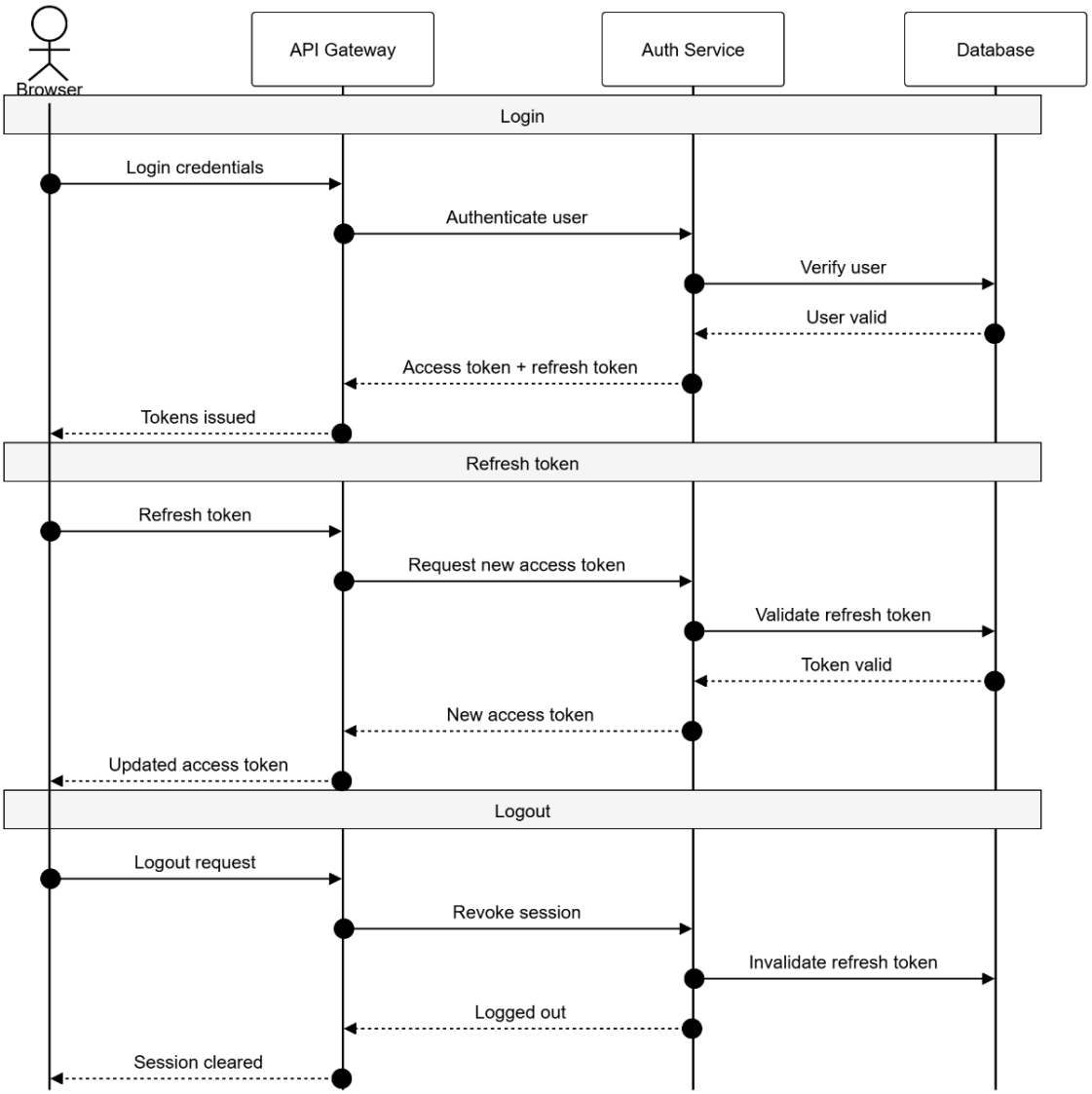


Figure 02: Authentication sequence - login, refresh, and logout. The browser sends credentials to the Auth Service via the API Gateway. On success, short-lived access tokens and longer-lived refresh tokens are issued. Refresh tokens are validated server-side and can be revoked on logout.

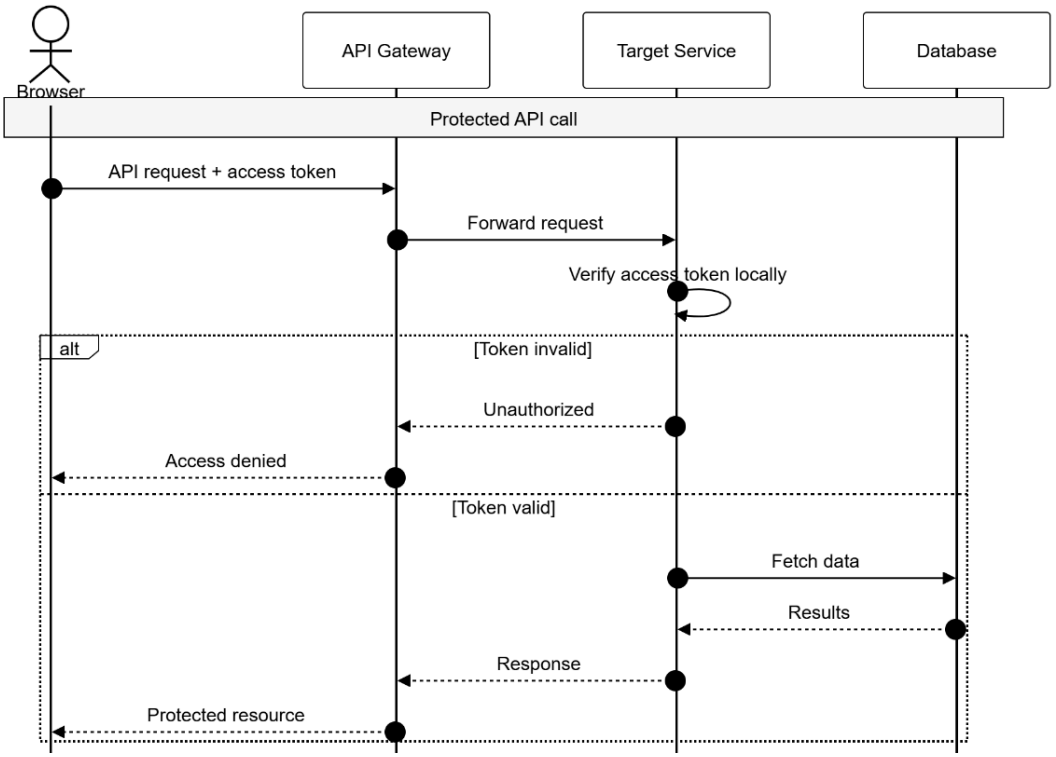


Figure 03: Authentication sequence - protected API call. The browser sends a request with an access token through the API Gateway to the target service. The service verifies the token locally; if valid, it fetches data and returns the resource. If invalid, the client receives an access-denied response. The Auth Service is not called on every request.

3.4.2 Authorization Model (RBAC)

Authorization is enforced via **role-based access control** at the NestJS controller level using RolesGuard (NFR-14). Each user holds exactly one role (FR-08).

Role	Key permissions	Restrictions
Guest	Browse catalogue; filter/NL search (FR-55)	Cannot save favourites (FR-56); cannot upload
Buyer	Search; favourites; profile (FR-16, FR-17, FR-10)	Cannot upload or administer
Dealer	Manage own listings; upload (Business tier); poll job status (FR-57–FR-59)	Cannot access admin routes; cannot modify other dealers' data
Administrator	User management; upload monitoring; audit logs; reports (FR-44–FR-49)	Cannot self-register (FR-12); actions logged (FR-47)

Dealer tier enforcement:
Bulk upload endpoints additionally check dealer_profiles.dealer_type = business and verification_status = verified before accepting files (ADR-006).

Capability	**Guest**	**Buyer**	**Dealer**	**Administrator**
Browse listings and search	**Yes**	**Yes**	**Yes**	**Yes**
Save favourites	**-**	**Yes**	**Yes**	**Yes**
Manage own profile	**-**	**Yes**	**Yes**	**Yes**
Create and manage own listings	**-**	**-**	**Yes**	**-**
Bulk inventory upload	**-**	**-**	**Business tier***	**-**
View upload job status	**-**	**-**	**Yes**	**Yes**
Verify dealers and manage users	**-**	**-**	**-**	**Yes**
View audit logs and reports	**-**	**-**	**-**	**Yes**

***bulk upload requires an approved business-tier account**

3.4.3 Infrastructure Security (IAM)

Each Lambda function receives a **least-privilege IAM execution role** (NFR-17). Illustrative scoping:

Lambda / Service	**S3**	**RDS**	**SQS**	**SES**	**Step Functions**	**Secrets**
Ingest API	Write raw bucket	Write ingestion schema	Publish	-	-	Read DB + JWT keys
ETL load function *	Read staging	Write ingestion; write marketplace.vehicles (column-scoped, via MarketplaceVehiclesWriteAdapter only)	-	-	-	Read DB
ETL embed function / images function	Read/write staging	-	-	-	-	-

Marketplace HTTP	Read processed images	Read/write marketplace schema	-	-	-	Read DB + Groq key
Admin HTTP	-	Read all schemas (SELECT)	-	-	-	Read DB
Notification	-	Write notification schema	Consume events	Send	-	Read SES config

No Lambda receives * permissions on production resources. Groq API key stored in Secrets Manager / SSM (NFR-20); never hard-coded.

The ETL load function is the one Lambda with a write grant outside its owning schema: a column-scoped INSERT/UPDATE grant on marketplace.vehicles, confined in code to MarketplaceVehiclesWriteAdapter (see section3.3.2 / ADR-002). No other cross-schema write grants exist.

3.4.4 Data Security

- **Passwords:** Salted hashes only (FR-07, NFR-15); never logged.
- **Transport:** All client-backend traffic over HTTPS (NFR-16).
- **Vehicle images:** Not publicly writable; served via **signed S3 URLs** (NFR-19).
- **Dealer verification documents:** Stored in private S3 prefix (dealer-verification/); lifecycle expiry per data-minimization (Data View Section 9).
- **Input validation:** All API bodies validated and sanitized (NFR-18); LLM outputs whitelist-validated (FR-21.3, FR-33).
- **Audit trail:** Sensitive admin actions recorded in admin.audit_logs with actor identity and timestamp (FR-47). Dealer approval: Admin Service calls Auth API; Admin Service writes the audit record - Auth Service does **not** write to admin.audit_logs directly.

3.4.5 External Dependency Security

Groq API is the **only non-AWS runtime dependency**. The architecture assumes Groq may be unavailable (rate limits, timeouts, outages). All Groq calls are wrapped in Retry/Catch with rules-only fallback (ADR-004, FR-42.1, NFR-12.1). No dealer or buyer PII is sent to Groq beyond unresolved field values and row descriptions required for normalization (FR-33.5, FR-33.7).

3.5 API and Interface Architecture

This section describes the system's external and internal API boundaries, complementing SRS Section 3.9 (Interfaces).

3.5.1 Public API Gateway (North-South)

All browser-to-backend traffic enters through a single **public Amazon API Gateway** REST endpoint. Routes are prefixed by service domain:

Route prefix	Target service	Auth required	Notes
/auth/*	Auth & User	Mixed (register/login public)	JWT issuance, refresh, dealer registration
/marketplace/*	Marketplace	Mixed (search public)	Listings, search, favourites
/ingest/*	Ingestion & ETL	Dealer (Business, verified)	Upload intake, returns 202 + Job ID
/jobs/*	Ingestion & ETL	Dealer	Upload job status polling (FR-32)
/admin/*	Admin	Administrator	Dashboard, users, audit logs, reports
/notifications/*	Notification	Internal/event-driven	Not browser-facing in MVP

Upload contract (Bulk Upload Intake):

- Request: POST /ingest/upload - multipart CSV/JSON + ZIP
- Success response: 202 Accepted + { jobId: uuid } (FR-31)
- Failure (format check): 400 Bad Request + plain-language reason; no job created
- Client polls: GET /jobs/{jobId} until status is completed or failed

3.5.2 Internal API Gateway (East-West)

Service-to-service calls use a **private internal API Gateway / ALB** (ADR-005):

Caller	Callee	Endpoint (illustrative)	Purpose
Admin Service	Auth Service	POST /internal/dealers/{id}/approve	Dealer verification approval
Admin Service	Auth Service	POST /internal/dealers/{id}/reject	Dealer verification rejection
Admin Service	Auth Service	POST /internal/users/{id}/deactivate	Account deactivation (FR-11)

Internal routes require a valid Administrator JWT or a service-to-service credential (implementation choice: mTLS or signed service token, document chosen approach at implementation time).

3.5.3 Asynchronous Interfaces

From	To	Mechanism	Payload
Ingest API	Step Functions	SQS message	{ jobId: uuid }
Step Functions	ETL Lambdas	Task state invocation	S3 key references
ETL pipeline	Notification Service	Event (via notify function or EventBridge)	Job summary counts
Notification Service	Amazon SES	AWS SDK	Rendered email

Dealer job status uses **polling** (FR-32, Section 3.9.4); no WebSocket in MVP.

Note: the ETL pipeline's Load stage does not communicate with Marketplace via an API or async interface, it writes directly into marketplace.vehicles under a scoped database grant (see section3.3.2, ADR-002). This is a deliberate exception to the interface-based communication pattern used elsewhere.

3.6 Error Handling and Resilience

3.6.1 ETL Pipeline Resilience

Failure type	Handling	SRS / ADR reference
Groq rate limit / timeout / malformed JSON	Retry with backoff; fall back to rules-only normalization for affected rows	FR-42.1, ADR-004
Row fails validation after rules + AI	Write to ingestion.rejected_records with reason; continue processing chunk	FR-42.2, FR-34
Transient Lambda / network failure	Step Functions Retry with exponential backoff	FR-42, NFR-06
Repeated SQS processing failure	Route to Dead-Letter Queue	FR-42.3
Duplicate SQS delivery	Idempotent Load logic keyed on upload job + row identity	FR-41
Entire chunk failure	Other chunks continue independently (Map state isolation)	Section 6.1 Process View

Graceful degradation example: A 1,000-row upload during Groq outage completes at ~95% success (rules-only normalization), with incomplete rows in rejected_records rather than total batch failure (Section 11.1).

3.6.2 Search Resilience

Failure type	Handling	Reference
Groq unavailable during NL search	Proceed with rules-parsed filters only; skip semantic ranking if no semantic text remains	NFR-12.1, FR-24
Query resolves entirely to structured filters	Skip vector embedding; order by price/date	NFR-12.1
Semantic ranking unavailable	Fall back to filter + pg_trgm search	FR-24
Zero results after full parse	Automated relaxation: widen numeric ranges or drop non-essential filters before empty state	Section 4.1.4

3.6.3 Notification Resilience

Notification Service retries transient SES failures without duplicate-sending a notification that already succeeded (FR-54). Delivery status recorded in notification.notifications (FR-53).

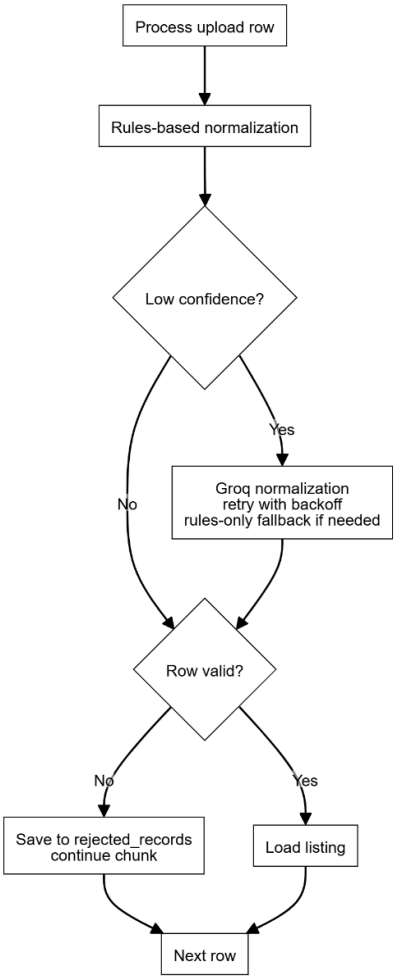


Figure 04: Per-row normalization, Groq fallback, and rejected-record handling.

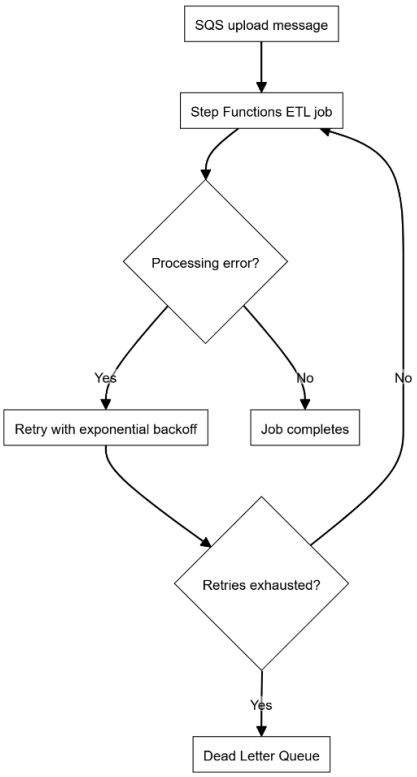


Figure 05: SQS-triggered ETL job with retry and Dead Letter Queue on repeated failure.

3.7 CI/CD and Environment Architecture

3.7.1 Repository Structure

Repository	**Deployable unit**	**Primary artifact**
auth-user-service	Auth Lambda(s)	Docker image -> ECR
marketplace-service	Marketplace Lambda(s)	Docker image -> ECR
ingestion-service	Ingest API + 11 ETL Lambdas	Docker image(s) -> ECR
admin-service	Admin Lambda(s)	Docker image -> ECR
notification-service	Notification Lambda(s)	Docker image -> ECR
web-frontend	React SPA	Static build -> S3 + CloudFront
cloud-infrastructure	Terraform modules	AWS resources
database/ (folder or repo)	Migrations	SQL migration scripts

Database migrations are **centralized** (Implementation View Section 8.1) because one physical RDS instance holds cross-schema foreign keys.

3.7.2 CI/CD Pipeline (GitHub Actions)

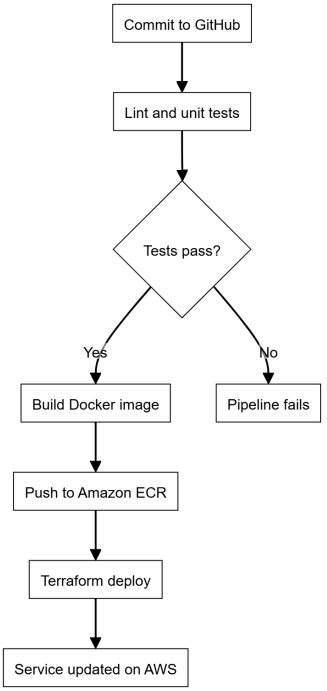


Figure 06 :Row-level ETL error handling. Each upload row is normalized with rules first; low-confidence rows use Groq with retry and rules-only fallback. Valid rows are loaded; invalid rows are saved to rejected_records and processing continues

Typical pipeline stages per service repository:

1. **Lint & typecheck** : ESLint, Prettier, TypeScript (NFR-21)
2. **Unit tests** : Jest; minimum coverage target per NFR-23
3. **Build** : Docker image for Lambda handlers
4. **Push** : Amazon ECR
5. **Deploy** : Update Lambda function image/tag via Terraform or AWS CLI
6. **Integration tests** (optional post-deploy) : smoke test against dev environment

Infrastructure changes flow through cloud-infrastructure Terraform plan/apply, never manual AWS Console changes (SRS Section 2.4, NFR-30).

3.7.3 Environments

Environment	Purpose	Constraints
Local (Docker Compose)	Developer testing	Internal HTTP between services; local Postgres or RDS dev instance

AWS Dev / Demo	Integration, UAT, viva demo	Free Tier limits; single RDS, single region
Production (future)	Not in MVP scope	Would add multi-AZ RDS, WAF, separate accounts

Local development mirrors east-west HTTP via Docker Compose service names, matching ADR-005.

3.8 Testing Architecture

This section describes the **architectural approach to testing**, not a detailed test plan. Testing validates that each architectural boundary (service, schema, pipeline stage) behaves as specified in the SRS.

3.8.1 Testing Layers

Layer	**Scope**	**Tools / approach**
Unit	Individual services, parsers, confidence scoring, adapters	Jest; mock repositories and AWS SDK adapters
Integration	Service + PostgreSQL; Service + S3; ETL stage handlers	Test containers or dedicated test schema; LocalStack for S3/SQS where feasible
Pipeline	End-to-end ETL: upload -> normalize -> embed -> load -> pending_review	Step Functions local or dev AWS; fixture CSV/ZIP files (SRS Appendix B)
End-to-end	Browser -> API Gateway -> full upload + search flow	Playwright or Cypress against dev deployment
Search quality	NL parser accuracy; embedding consistency	Fixture queries; assert FR-22.1 model parity

3.8.2 Architecturally Significant Test Cases

- **ADR-002:** Load adapter writes only to marketplace.vehicles and vehicle_images; DELETE attempt fails at DB grant level.
- **ADR-004:** Row with confidence ≥ 0.6 never invokes Groq; row below 0.6 does; Groq outage falls back to rules-only.
- **ADR-006:** Individual dealer receives 403 on bulk upload endpoint; Business verified dealer receives 202.
- **FR-41:** Reprocessing same SQS message does not create duplicate listings.
- **FR-42:** ETL-loaded listing has status = pending_review; not returned in public search until dealer approves.

- **FR-22.1:** Query embedding model matches listing embedding model (same library version).

3.8.3 Test Data

SRS Section 2.2 uses an artificial vehicle data generation tool that allows tests without any dealer data. Tests fixtures comply with SRS Appendix B (clean row + dirty row) and are used for validating deterministic parser vs. Groq fall-back paths.

4. Use-Case View

The following use cases describe the important architectural features of the system. Each one uses more than one service and/or a delicate architectural component (asynchronous processing, hybrid search ranking, or human review gate).

Guest Use Cases

The guest interacts with the system through the public-facing web frontend without requiring authentication.

Use Case	Brief Description
Register	Allows the guest to create a new account to become an authenticated Buyer or Dealer.
Search Vehicles	Allows the guest to search the marketplace using traditional UI filters or free-text natural language queries.
View Vehicle Details	Displays the full specifications, image gallery, and dealer contact information for a specific vehicle listing.

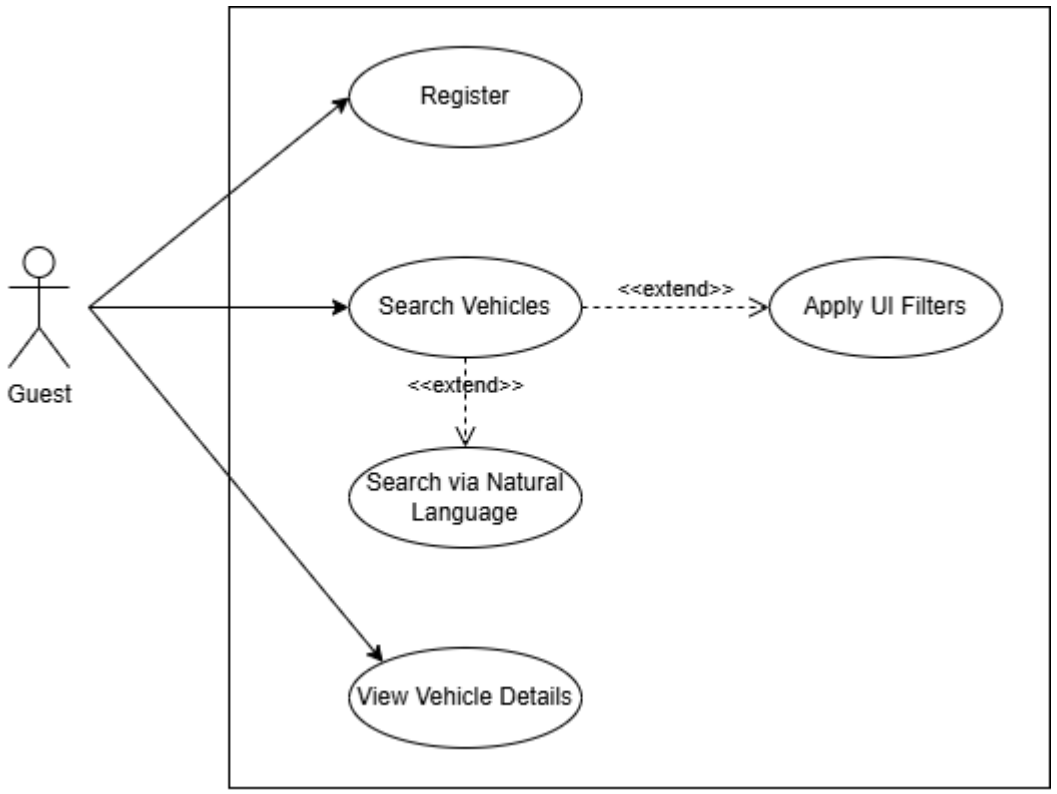


Figure 7 : shows the main functions available to an unauthenticated guest. The architecturally significant guest function is the natural-language vehicle search, as it triggers the 5-stage deterministic parser, interacts with the external Groq LLM API for fallback resolution, and queries persistent vector embeddings using PostgreSQL's pgvector

Buyer Use Cases

The buyer interacts with the system through the authenticated web frontend.

Use Case	Brief Description
Log In	Authenticates the buyer and provides a JWT for access to secured application features.
Search Vehicles	Allows the buyer to search listings using structured UI filters or natural language, extending the guest capability.
View Vehicle Details	Displays the full vehicle information and dealer details.
View Recommendations	Displays semantically related vehicle listings based on vector similarity.
Manage Profile	Allows the buyer to view and update their personal profile information.

Manage Favourites	Allows the buyer to save specific vehicle listings to a personal list and remove them as needed.

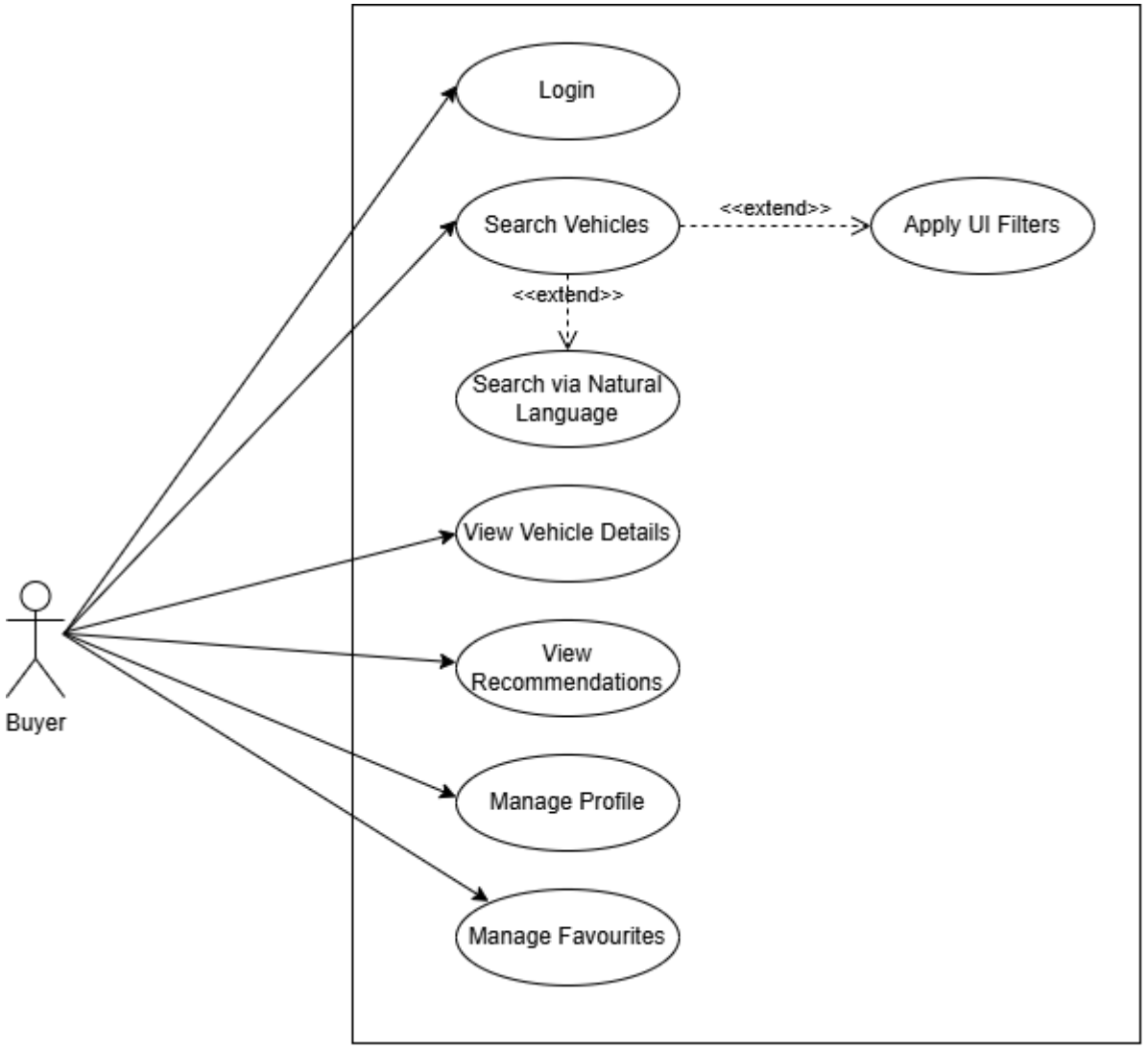


Figure 8 : the functions available to a buyer. The "Manage Favourites" and "View Recommendations" use cases have significant architectural coverage, as they bridge authentication boundaries, require foreign-key interactions between the auth and marketplace database schemas, and heavily utilize semantic similarity calculations.

Dealer Use Cases

The dealer interacts with the system primarily through the authenticated dealer dashboard on the web frontend.

Use Case	Brief Description
Register	Submits a dealer registration request, which is placed in a pending state until approved by an Administrator.

Log In	Authenticates the dealer and provides access to inventory management functions.
Upload Inventory	Allows the dealer to perform a bulk upload using a CSV/JSON file alongside a ZIP archive of vehicle images.
View Dashboard	Displays an overview of the dealer's active inventory, upload history, and profile status.
Track Upload Status	Allows the dealer to monitor the progress of active bulk upload jobs and view row-level validation errors.
Manage Listings	Allows the dealer to edit details, deactivate live listings, and explicitly review/confirm any data that was automatically generated by the LLM (Pending Review status).
Receive Notifications	Delivers asynchronous email alerts to the dealer regarding the success or failure of their bulk upload jobs.

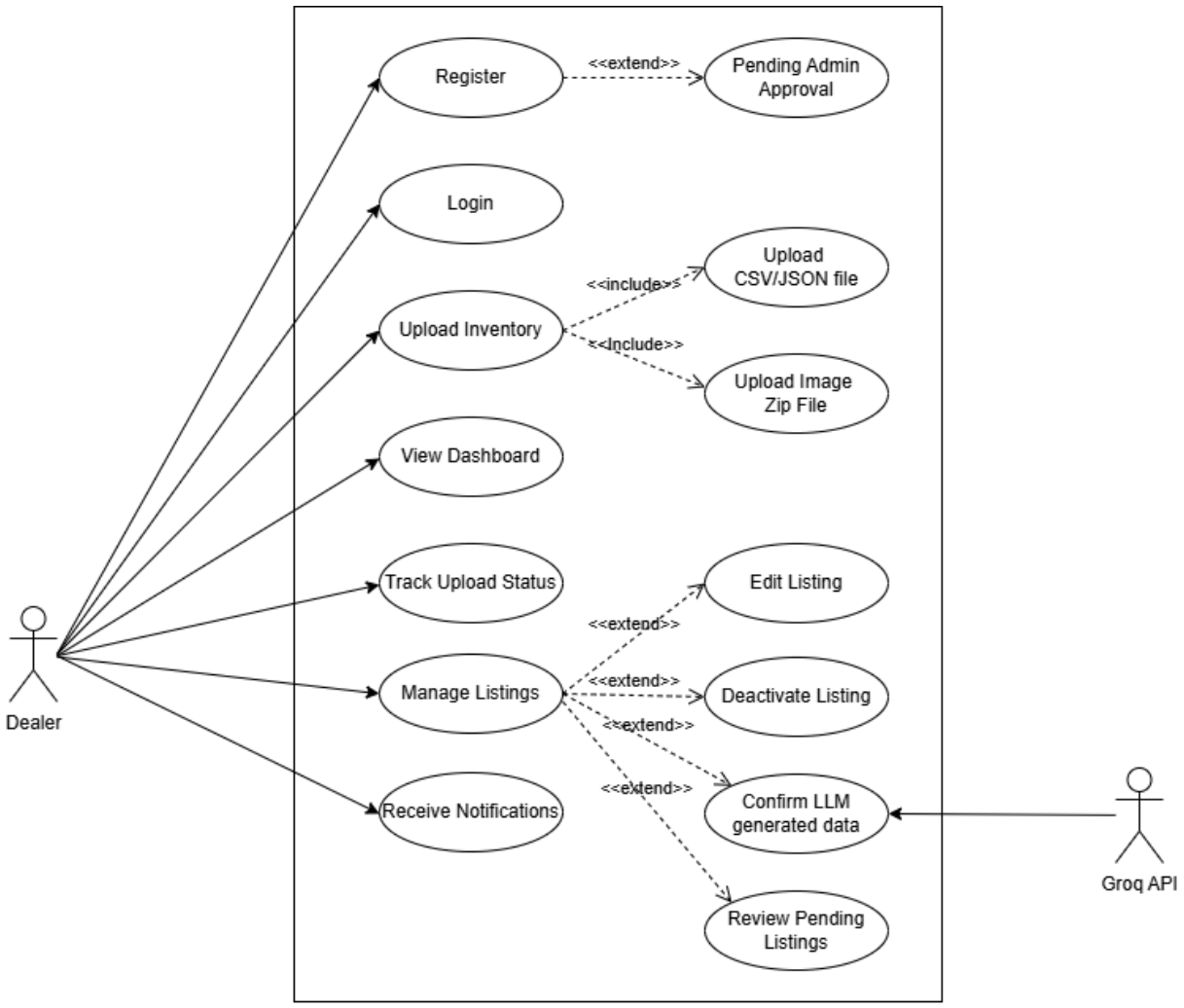


Figure 9 : details the functions available to a dealer. Inventory upload and listing management are the most architecturally complex operations in the system. They orchestrate the 11-Lambda AWS Step Functions ETL pipeline, handle distributed file processing via Amazon S3 and SQS, enforce human-in-the-loop AI validation, and execute the system's single cross-schema write exception into the marketplace.vehicles table.

Administrator Use Cases

The administrator uses a secured administrative web dashboard to manage institution-level information, user statuses, and overall platform health.

Use Case	Brief Description
Log In	Authenticates the administrator and provides access to system-wide administrative functions.
Review Audit Logs	Allows the administrator to review chronological records of sensitive system and administrative actions.
Monitor Upload Jobs	Displays the status and metrics of all ETL upload jobs occurring across the entire platform.
View Reports	Provides platform-level analytical summaries regarding total listings, user activity, and ETL error rates.
Manage Users	Allows the administrator to view all user accounts, approve pending dealer registrations, and suspend or reactivate profiles.

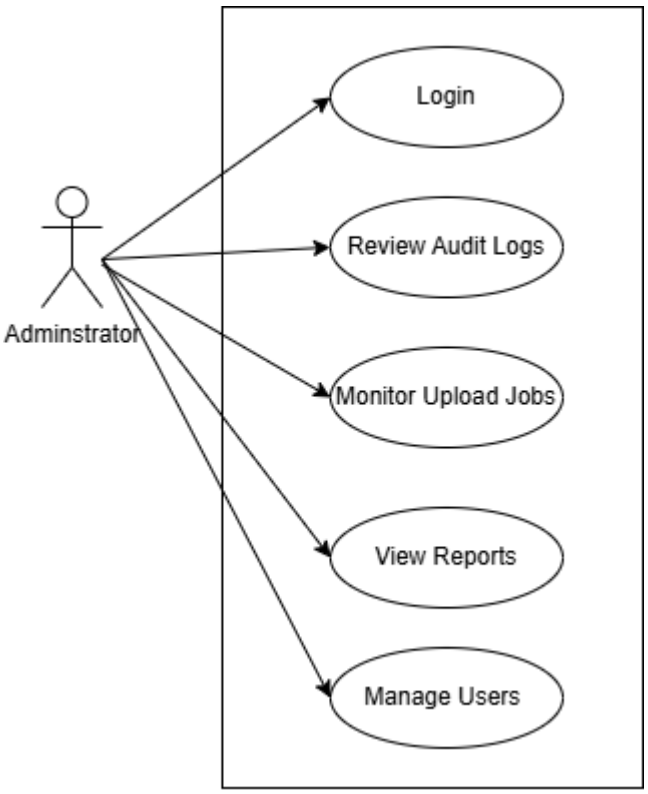


Figure 10: shows the administrative functions required to oversee the platform. Dashboard reporting and monitoring use cases are architecturally significant because they exercise the system's admin_service_role database isolation exception, safely executing cross-schema read queries to aggregate data from the auth, marketplace, and ingestion schemas without compromising service write boundaries.

4.1 Use-Case Realizations

The following realizations describe selected use cases that exercise several architectural elements or represent operationally important workflows. Supporting activity, sequence, and state diagrams for these realizations are presented later in the Process View.

4.1.1 Register and Tiered Authorization

Use case name	Register and Tiered Authorization
Actor(s)	Guest, Dealer, Administrator
Description	A Guest registers as a Dealer, selecting a specific tier (Individual or Business) and uploading required documentation. An Administrator reviews the application, unlocking tier-specific platform capabilities upon approval.

Preconditions	The supplied email address is not associated with an existing account. The required document (NIC for Individual; Business Registration Certificate for Business) is provided.
Main flow	1. The Guest submits registration details, selects a dealer type, and uploads the required documentation via the frontend. 2. The Auth-User Service hashes the password and stores the uploaded documents in a private Amazon S3 prefix (dealer-verification/). 3. The service creates a new dealer_profiles record in PostgreSQL (auth schema) with a pending verification status. 4. An Administrator reviews the uploaded documents via the secure Admin dashboard. 5. The Administrator approves the application. 6. The Auth-User Service updates the database status to verified, logs the decision to audit_logs, and unlocks platform capabilities based on the dealer type (manual listing limits for Individual; bulk upload access for Business). 7. The dealer is notified of the approval via email.
Successful end/post condition	A verified dealer_profiles row exists, tier-specific platform capabilities are unlocked, and the client holds a valid JWT with appropriate role claims.
Fail end/post condition	If the Administrator rejects the application, the status becomes rejected, the reason is logged, and the dealer is notified to resubmit corrected documents.
Contributing design elements	Auth-User Service, Amazon S3, PostgreSQL (auth schema), Notification Service (Amazon SES).

4.1.2 Bulk Upload Intake

Use case name	Bulk Upload Intake
Actor(s)	Dealer

Description	A verified Business Dealer submits a CSV/JSON inventory file and a ZIP of vehicle images. The system stores the raw files and starts asynchronous processing without blocking the dealer's request.
Preconditions	The Dealer is authenticated, possesses a valid JWT, and has an approved Business-tier profile.
Main flow	1. The Dealer submits the CSV/JSON file and image ZIP via the frontend. 2. The Ingest API (Ingestion and ETL Service) verifies the JWT locally against a shared public key, avoiding a network call to the Auth Service. 3. The API performs a format-only check (validating it is parseable CSV/JSON and a real ZIP archive). 4. The API streams the raw files, untouched, directly into Amazon S3. 5. The API creates an upload_jobs row in PostgreSQL (ingestion schema) with a PENDING status. 6. The API publishes exactly one message containing the Job ID to an Amazon SQS queue. 7. The API immediately responds to the frontend with a 202 Accepted HTTP status and the Job ID.
Successful end/post condition	An upload_jobs record exists with status PENDING, raw files are durably stored in S3, and the payload is queued in SQS awaiting Step Functions execution.
Fail end/post condition	If the files fail the format-only check, the request is rejected synchronously with a clear reason, and no job or queue message is created.
Contributing design elements	Ingestion and ETL Service, Amazon S3, Amazon SQS, PostgreSQL (ingestion schema).

4.1.3 Process and Review Pending Listings

Use case name	**Process and Review Pending Listings**
Actor(s)	Dealer
Description	Following a bulk upload, an automated ETL pipeline orchestrates the chunking, normalization, and embedding of data using targeted AWS Lambda functions. All accepted rows are persisted as PENDING_REVIEW listings. The Dealer then completes the use case by explicitly reviewing and confirming any AI-inferred data before the listings are published.
Preconditions	The Dealer has successfully initiated the "Bulk Upload Intake" use case, and the asynchronous AWS Step Functions execution is underway.

Main flow	1. The validateFile function Lambda checks headers and file integrity once for the entire file. 2. The splitChunks function Lambda divides the file into 100-row chunks, passing S3 descriptor pointers to a parallel Map state. 3. Inside the Map state, the parseNormalize function Lambda applies a 6-step normalizer to each field and calculates a row-level confidence score. 4. If the confidence score is < 0.6, the groqNormalize function Lambda routes unresolved fields to the external Groq API (LLM) for contextual inference, subsequently dropping any hallucinated values. 5. The validateRows function Lambda validates mandatory business rules; rows failing this step are written to rejected_records. 6. The enrich function Lambda maps JSONB specifications based on vehicle type and builds the final search_text. 7. The embed function Lambda generates a 384-dimensional vector embedding using the all-MiniLM-L6-v2 model. 8. The Load function Lambda executes a bulk-insert, saving accepted rows into the marketplace.vehicles table with a PENDING_REVIEW status. 9. The Dealer opens the dashboard, reviews the AI-inferred fields, and confirms the data, transitioning the listing status to LIVE.

Successful end/post condition	Valid rows exist as LIVE searchable listings, rejected rows are recorded with actionable reasons, and the dealer has explicitly confirmed all AI-inferred data.
Fail end/post condition	If the Groq API is unavailable, the pipeline catches the error and proceeds to validation using rules-only data, degrading gracefully rather than failing the entire batch.
Contributing design elements	AWS Step Functions, ETL Lambdas (validateFile function, splitChunks function, parseNormalize function, groqNormalize function, validateRows function, enrich function, embed function, Load function), Amazon S3, Groq API, PostgreSQL, Marketplace Service.

4.1.4 Intelligent Search Execution

Use case name	**Intelligent Search Execution**
Actor(s)	Buyer, Guest
Description	A user searches the marketplace using a free-text natural-language query. The system utilizes a 5-stage deterministic parser, an LLM fallback, and semantic embeddings to return a highly accurate, ranked result list.
Preconditions	None. The feature is available to both authenticated Buyers and unauthenticated Guests.

Main flow	1. The user submits a free-text query via the frontend search bar. 2. The Marketplace Service tokenizes the query, masks stopwords, and executes Stage 1 (Multi-word phrase extraction) and Stage 2 (Numeric pattern extraction). 3. The service executes Stage 3 (Exact Dictionary Hits) and Stage 4 (Fuzzy dictionary hits via pg_trgm), strictly resolving Make before Model. 4. The service calculates a confidence score based on coverage and numeric signals. 5. If the confidence score is < 0.6, unresolved tokens and a shortlist of candidates are sent to the Groq API for LLM-assisted filter extraction. 6. All returned LLM fields are strictly whitelist-validated against the database schema. 7. The service passes the remaining unconsumed semantic text to the embedding library to produce a query vector. 8. The service executes a single PostgreSQL query combining exact filter conditions (WHERE clauses) and pgvector semantic ranking (ORDER BY embedding <=> vector). 9. Ranked, paginated results are returned to the user.
Successful end/post condition	A ranked list of matching listings is returned, and a record is logged in the search_queries table to fuel future alias promotion.
Fail end/post condition	If the query returns zero results, the system executes an automated relaxation path (widening numeric ranges or dropping non-essential filters) before returning an empty state.
Contributing design elements	Marketplace Service, Groq API, PostgreSQL (marketplace schema, utilizing pg_trgm and pgvector).

4.1.5 Monitor Platform Activity and Audit Logs

Use case name	Monitor Platform Activity and Audit Logs
Actor(s)	Administrator
Description	The Administrator logs into the web dashboard to monitor upload jobs across the platform, review system audit logs, and view aggregate reports.This scenario exercises the system's cross-schema read architecture, allowing the Admin Service to aggregate data securely across service boundaries
Preconditions	The Administrator is authenticated with a valid JWT containing Administrator role claims.
Main flow	1. The Administrator requests the platform dashboard view via the frontend. 2. The API Gateway routes the request to the Admin Service. 3. The Admin Service utilizes the admin_service_role database user to execute authorized cross-schema SELECT queries across the isolated auth, marketplace, and ingestion schemas. 4. The Admin Service retrieves aggregate bulk upload statuses and metrics directly from the ingestion.upload_jobs table. 5. The Admin Service retrieves records of sensitive system actions from its own admin.audit_logs table. 6. The Admin Service retrieves total active listing metrics from the marketplace.vehicles table. 7. The service formats the aggregated data and returns it to the client to populate the dashboard tiles.
Successful end/post condition	The Administrator receives an accurate, real-time aggregate view of platform activity, and the system successfully executes cross-schema reads without violating write-isolation rules.
Fail end/post condition	An unauthorized request (e.g., a Dealer attempting to access the route) is intercepted by role-based guards and rejected with a 403 Forbidden status, returning no data.
Contributing design elements	Admin Service, PostgreSQL (specifically utilizing the admin_service_role cross-schema SELECT grants).

5. Logical View

5.1 Overview

This architecture splits into five independently deployable services and one shared library layer. Each of the services is designed to use the same internal package structure (config, common, infrastructure, modules, health, lambda). So that each team member can move through each service easily. The domain logic resides in the modules/{feature}/{controllers,services,repositories,entities,dto} layer while the usage of the AWS SDK is done from infrastructure/adapters layer only.

- **auth-user-service** : registration (including tiered Individual/Business dealer verification), login, JWT/refresh tokens, roles, dealer approval.
- **marketplace-service :** listings, favourites, filter/NL/semantic/trigram search, recommendations, and the shared vehicle_dictionaries reference table (makes/models/fuel/transmission) used for trigram matching.
- **ingestion-service :** the Ingest API, Job Status API, and the eleven-Lambda Step Functions ETL pipeline.
- **admin-service :** user management, upload monitoring, reports, audit logs.
- **notification-service :** event-driven SES email notifications.
- **shared normalize + embed library :** the dictionaries, regex rules, and MiniLM embedding wrapper used by bo
- the marketplace-service (query-side) and ingestion-service (listing-side), per the model-parity constraint in Section 3.

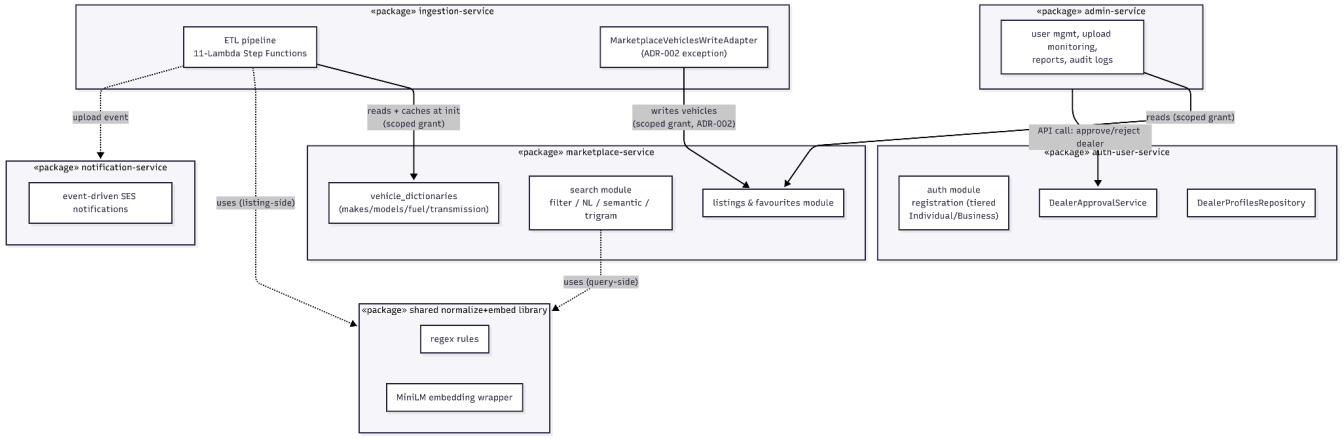


Figure 11: Backend module packages and documented cross-module dependencies (shared RDS, service-oriented modular architecture).

5.2 Architecturally Significant Design Packages

5.2.1 Auth & User - auth module

- **AuthController :** login/refresh/logout HTTP entrypoints, plus dealer registration (including document upload) entrypoints.

- **AuthService :**credential verification, JWT issuance, orchestrates RefreshTokenService; for Dealer registration, captures the selected dealer_type and stores verification document references via DealerProfilesRepository.
- **RefreshTokenService :** issues, validates, and revokes refresh tokens (hashed at rest).
- **JwtStrategy / RolesGuard :** Passport JWT validation and RBAC enforcement used by every service.
- **DealerApprovalService :** transitions dealer_profiles.is_verified through pending/verified/rejected; exposed to admin-service via the internal API Gateway/ALB rather than a direct database write from Admin.
- **DealerProfilesRepository :** persists dealer_type, business fields, verification_documents, and is_verified state; written by AuthService at registration and by DealerApprovalService at approval/rejection.

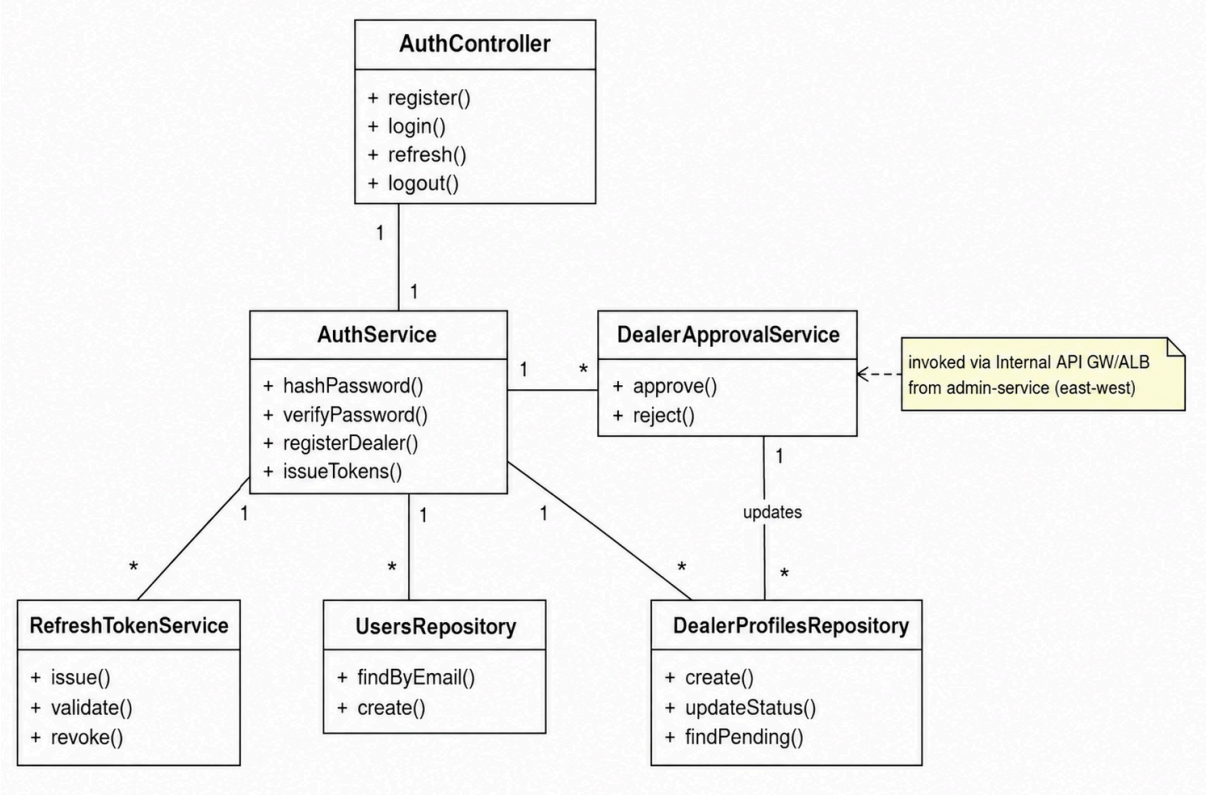


Figure 12: auth- user class diagram

5.2.2 Marketplace - search module

- **SearchController :** exposes filter, natural-language, and combined search endpoints.
- **DeterministicParser :** extracts structured filters from free text (shared logic family with the ingestion parser, per the ETL design's "same rules-first structure" principle).
- **PgvectorSearchAdapter / TrigramSearchAdapter :** thin infrastructure-layer wrappers around the respective PostgreSQL extensions.

- **SearchService :** merges filter, vector, and trigram results into one ranked response and applies the semantic-search fallback.
- **VehicleDictionariesRepository :** owns the vehicle_dictionaries reference table (canonical makes/models/fuel/transmission values plus alias history); read directly by ingestion-service via a scoped grant and cached in memory at container init.

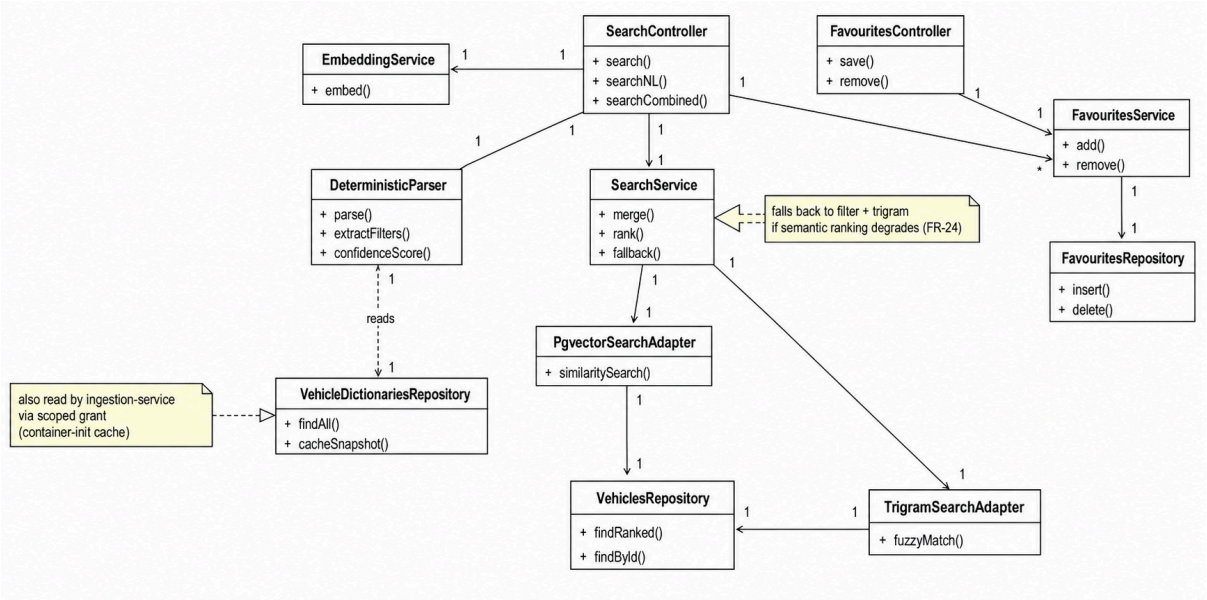


Figure 13 : Marketplace class diagram

5.2.3 Ingestion & ETL - pipeline package (src/workers/etl-worker/pipeline/)

- **normalize/ :** ParseNormalizeHandler (rules, confidence scoring) and GroqNormalizeHandler (LLM fallback, flagged rows only).
- **validate/ :** ValidateFileHandler (whole-file structural checks) and ValidateRowsHandler (per-row mandatory-field checks).
- **enrich/ :** EnrichHandler (derived fields, JSONB attributes, search_text construction).
- **embeddings/ :** EmbedHandler (MiniLM inference, loaded at module scope for warm-container reuse).
- **image-processing/ :** ImagesHandler (Sharp resize/compress, registration-number keyed matching to rows).
- **persistence/ : MarketplaceVehiclesWriteAdapter -** the only class with the scope and column-based granting of ingestion_service_role in marketplace.vehicles; plain parameterized SQL and not an import of shared entity classes, in order to keep the domain models of the two services separate. This is the only intentional exception to write ownership by each service (see Goal 3 in Cross-service data isolation model from Section 3 for more details); restricting it to only one named class ensures that any such future write to another schema will stand out in the diffs.

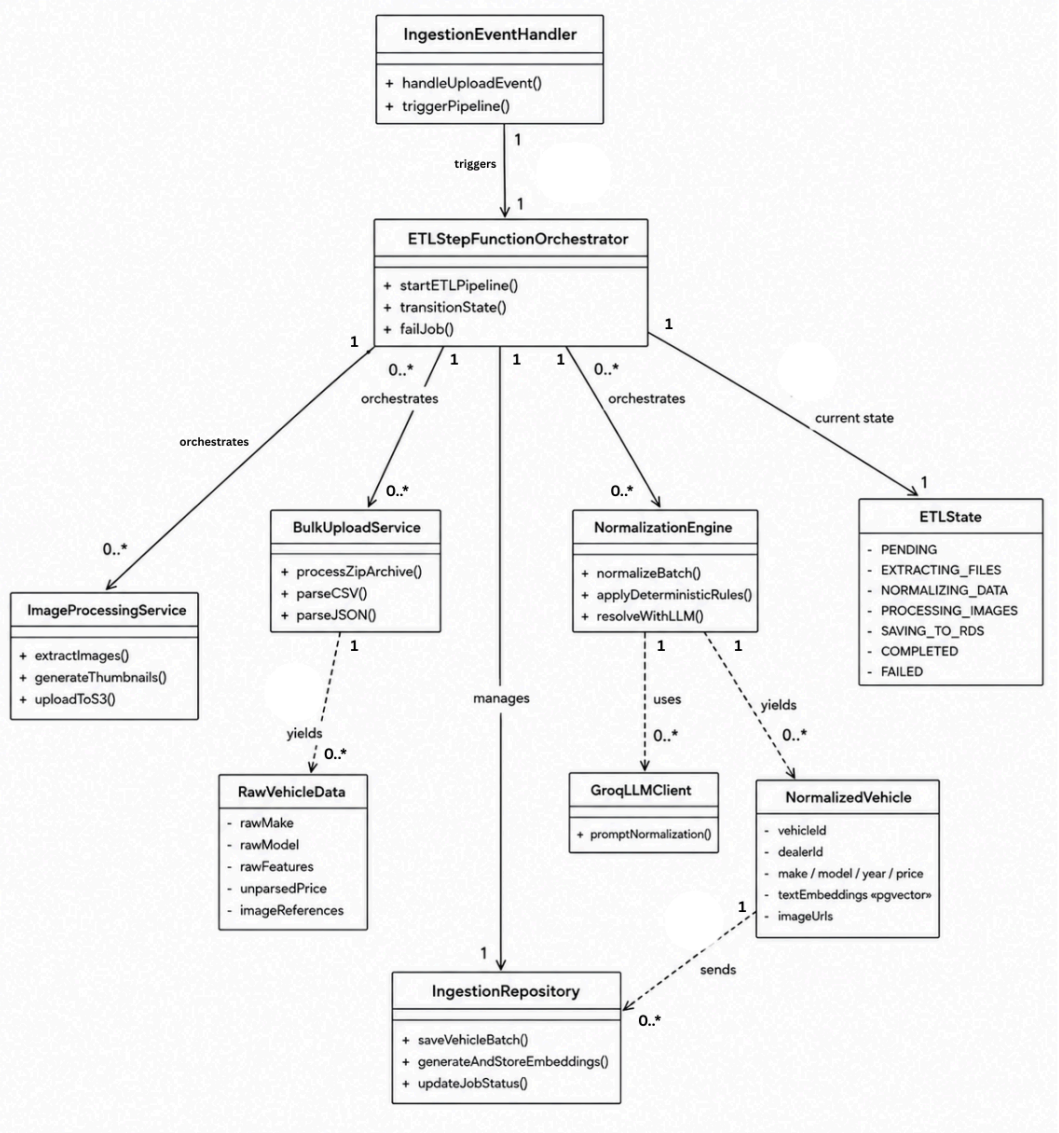


Figure 14 : Ingestion class diagram

5.2.4 Admin - modules

- **UserManagementService, ReportService, AuditLogService, DashboardService :** as scoped in the SRS's FR-43 through FR-48.

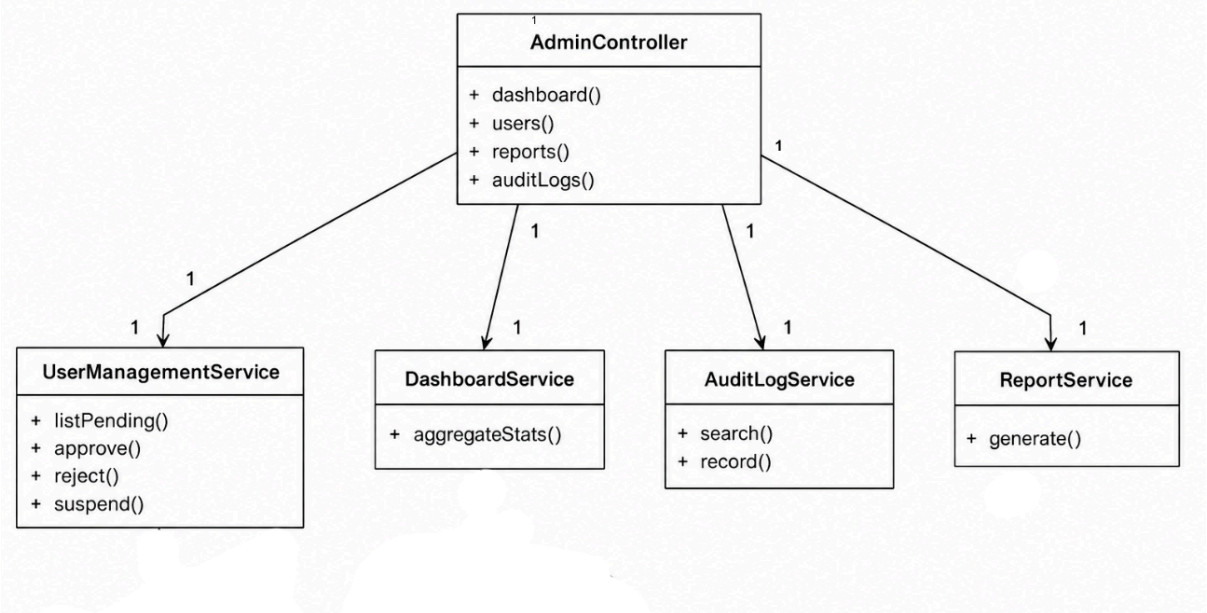


Figure 15 : Admin class diagram

5.2.5 Notification - notifications module

- **NotificationEventHandler, EmailTemplateService, SesAdapter :** as scoped in FR-49 through FR-53.

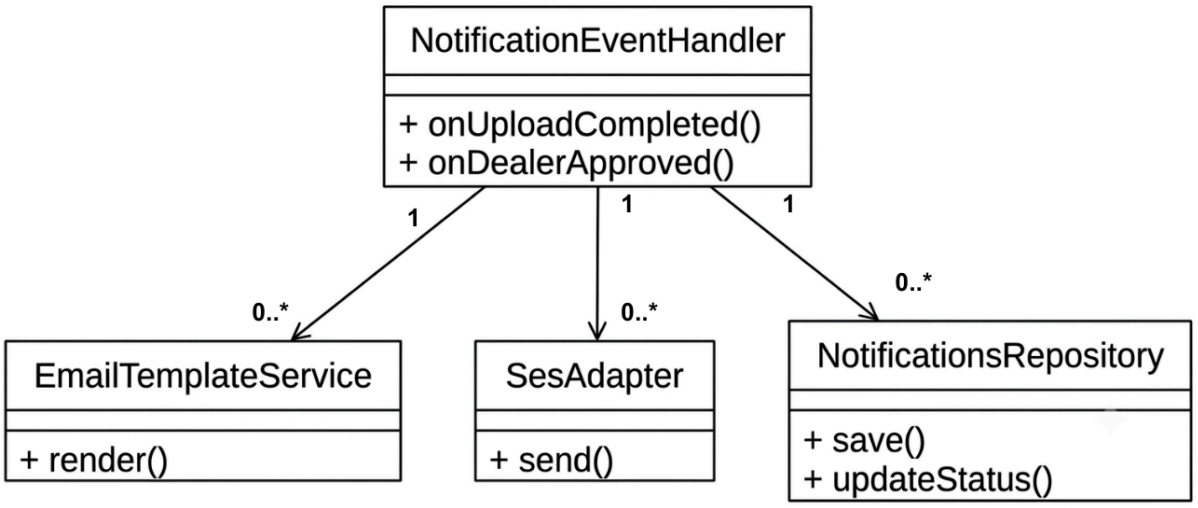


Figure 16: Notification class diagram

6. Process View

This part explains the process decomposition in terms of lightweight and heavyweight processes (threads of control). It explains the interactions among modules, the AWS infrastructure, and external APIs to achieve the main use cases for the Second Hand Vehicle Marketplace.

6.1 Process Organisation and Communication

The main runtime processes are organized into the following groups:

Process group	Main participants	Communication
Dealer Registration & Authentication	Dealer, React Frontend, API Gateway, Auth-User Service, PostgreSQL (auth schema), Amazon S3	Synchronous API requests, JWT local validation, and relational database queries.
Bulk Upload Intake	Dealer, React Frontend, API Gateway, Ingestion API, Amazon S3, Amazon SQS	Synchronous API requests for file ingestion, transitioning to asynchronous message passing via SQS queues.
ETL Chunk Processing	AWS Step Functions, ETL Lambdas (Normalize, Validate, Enrich, Embed, Load), Groq API, PostgreSQL (marketplace schema)	Orchestrated state machine transitions passing S3 reference keys, synchronous external LLM API calls, and optimized database batch inserts.
Hybrid Natural-Language Search	Buyer, React Frontend, API Gateway, Marketplace Service, Groq API, PostgreSQL (marketplace schema)	Synchronous API requests, fast in-memory parsing, external LLM fallback via HTTP, and semantic vector database queries.
Administrative Monitoring	Administrator, Web Dashboard, Admin Service, PostgreSQL (All schemas)	Synchronous authorized API calls executing cross-schema database reads via restricted PostgreSQL roles.

6.2 Bulk Inventory Upload & ETL Orchestration Activity

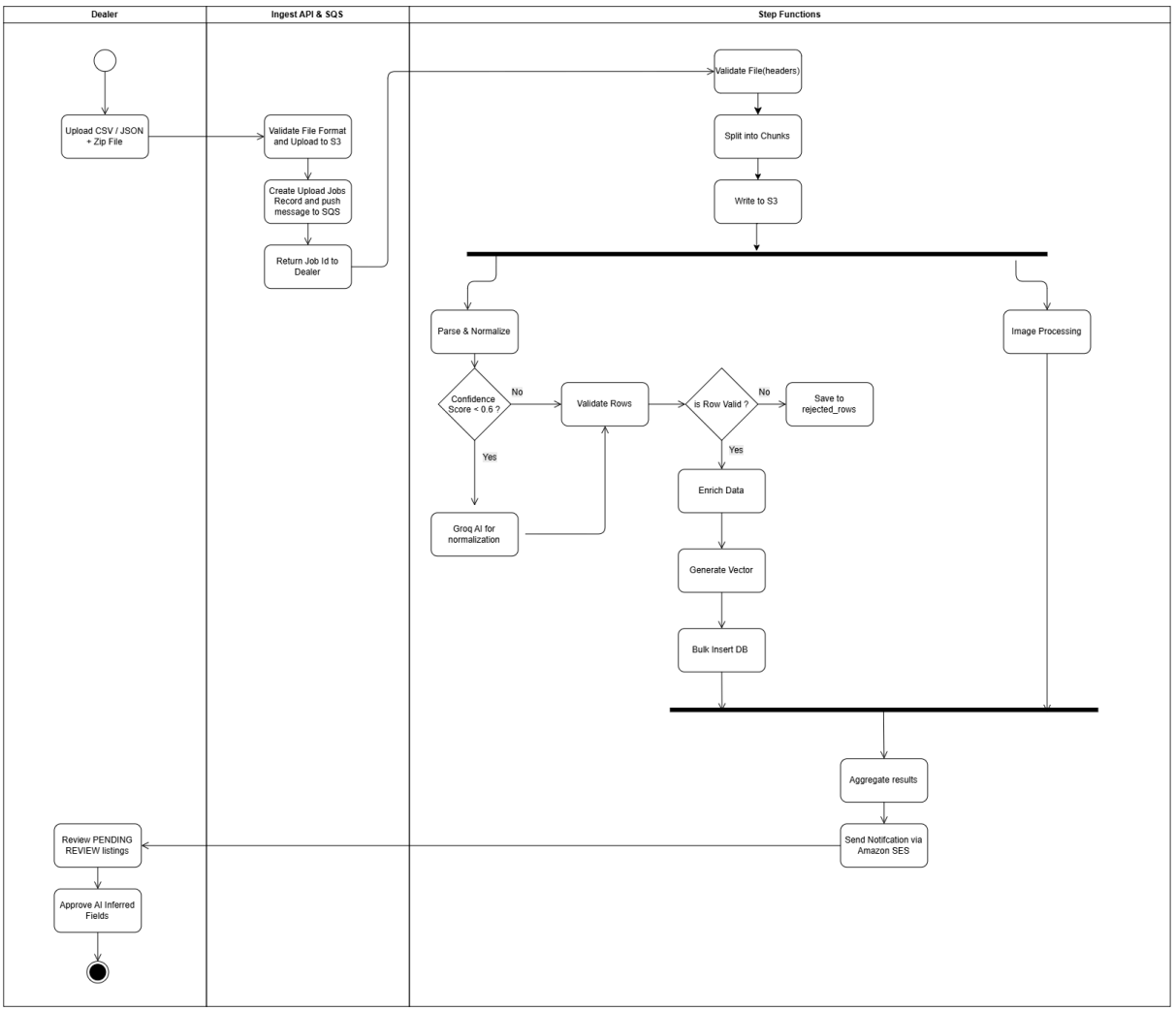


Figure 17: shows the end-to-end workflow of bulk inventory upload from the perspective of the Dealer, Ingest API, AWS Step Functions, and SQS swimlanes. The dealer uploads a CSV or JSON file that contains data about vehicles together with a ZIP file of vehicle images. As the aim is to provide a synchronous response at a rapid pace, the Ingest API only runs a lightweight validation, keeps the files unmodified in the S3 storage service of Amazon, and kicks off an SQS queue with a message.

The extraction, transformation, and loading process is controlled by an AWS Step Functions workflow. Initially, the workflow conducts header validation and splits the dataset into chunks. Chunks are processed in parallel using a Map state. In each of the Map states, the system conducts normalization, uses Groq LLM to validate low-confidence fields, validates the data, generates vector embeddings, and inserts data into the database in bulk. Listings that are processed correctly are marked as PENDING_REVIEW which implies that they have to be confirmed by the dealer. Meanwhile, another branch processes vehicle images and adds them to listings.

6.3 Hybrid Natural-Language Search Activity

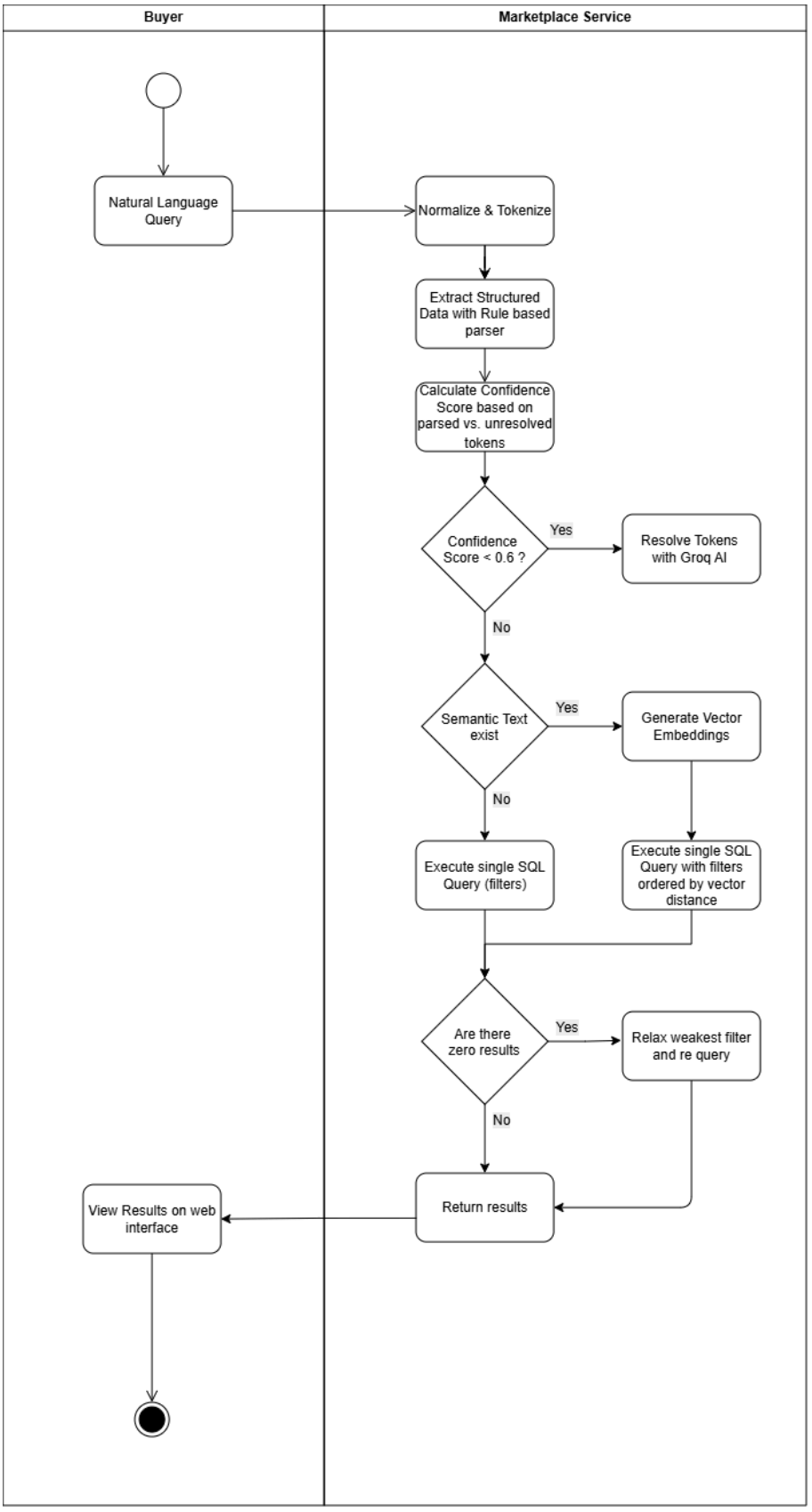


Figure 18: describes how a buyer's free-text query is processed by the Marketplace Service. The query is tokenized and passed through a multi-stage deterministic parser that extracts multi-word

phrases, numeric patterns, and exact dictionary matches. The parser utilizes pg_trgm to perform fuzzy similarity matching for large vocabularies like vehicle makes and models.

In case the parser generates a confidence level that falls short of an acceptable value, the unparseable tokens are sent over to the Groq API to be analyzed using LLMs. The unparseable descriptive words get mapped to the semanticText and converted to a vector with a dimension of 384. Lastly, a single PostgreSQL query is run to execute the search.

6.4 Dealer Registration & Tiered Authorization Sequence

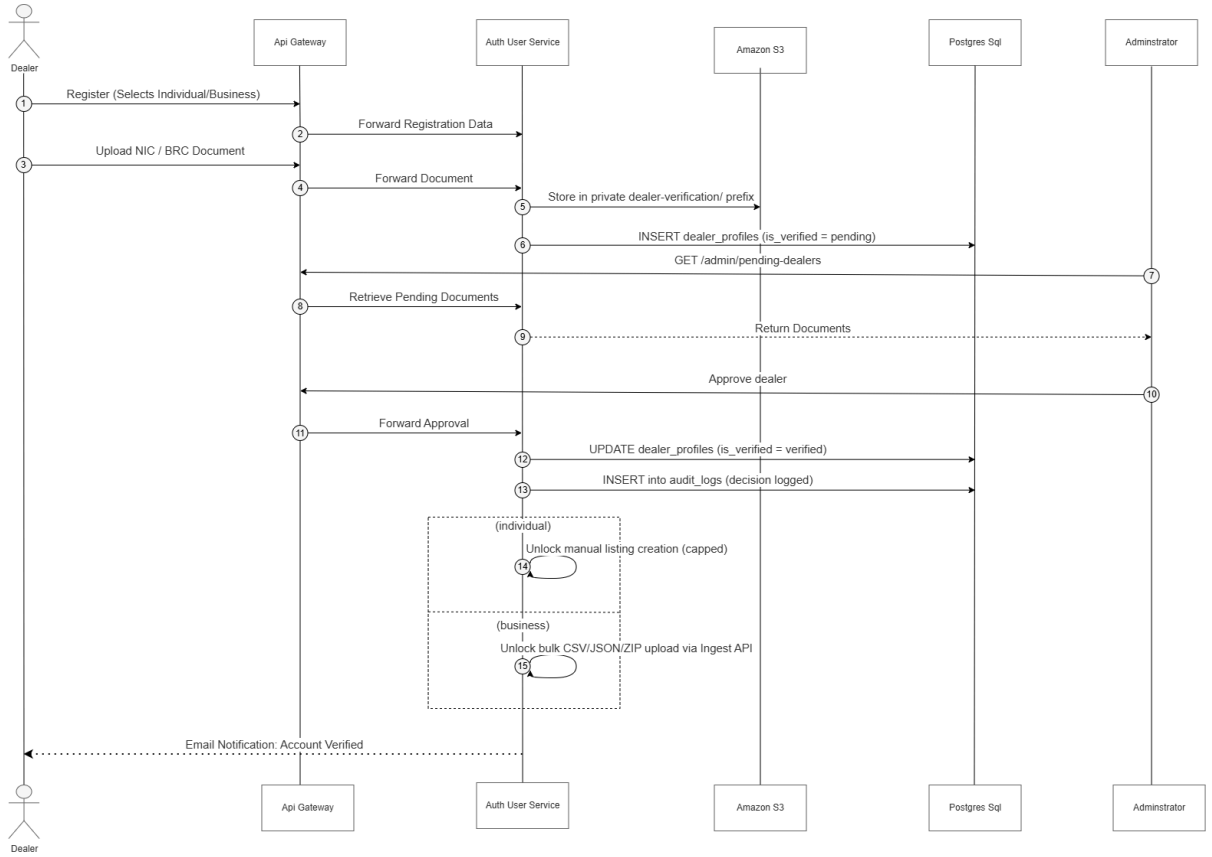


Figure 19: shows how the registration, validation, and authorization for a Dealer are accomplished via the Auth-User service. During registration, the dealer is required to enter a dealer type, which can be either Individual or Business, and provide the required supporting document, which can be NIC and/or Business Registration Certificate. The provided documents are stored in a private bucket on Amazon S3.

The account gets into the pending mode until the documents get approval by an administrator through the Admin dashboard. Once the documents are approved, the approval is logged into the audit log, and certain features would get unlocked based on the dealer type; individual dealers are able to make manual listings with limited numbers, while business dealers are able to upload listings in bulk through CSV/JSON/ZIP files using Ingest API.

6.5 Bulk Upload Intake & SQS Decoupling Sequence

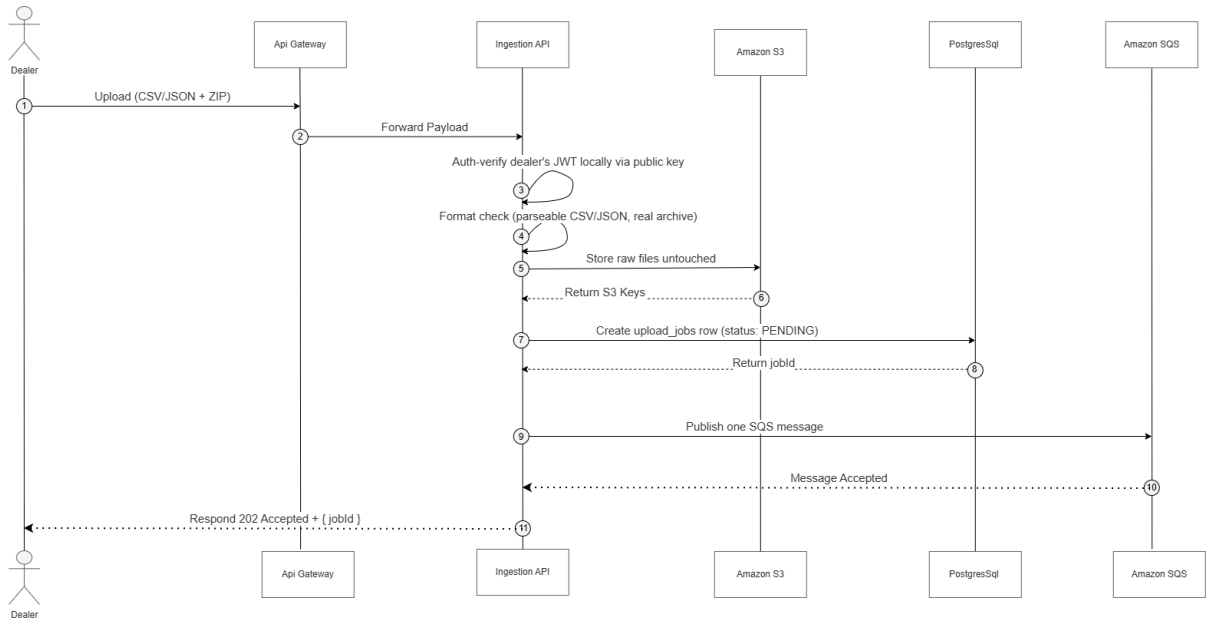


Figure 20: shows the asynchronous intake boundary covered by the Ingestion and ETL Service. For ensuring that the response time is fast and the process happens in a synchronous manner, the time-consuming task is offloaded to the AWS Step Function state machine.

The Ingest API just checks if the token from the dealer is JWT or not through the use of the public key, which is done without making any calls to the Auth Service for each individual request. The basic validation for ensuring that the payload has the required format (CSV/JSON in ZIP file) is also performed by the API. The raw files are kept intact on Amazon S3 for being the point of truth. An upload job and one SQS message are created.

6.6 ETL Chunk Processing (The Map State) Sequence

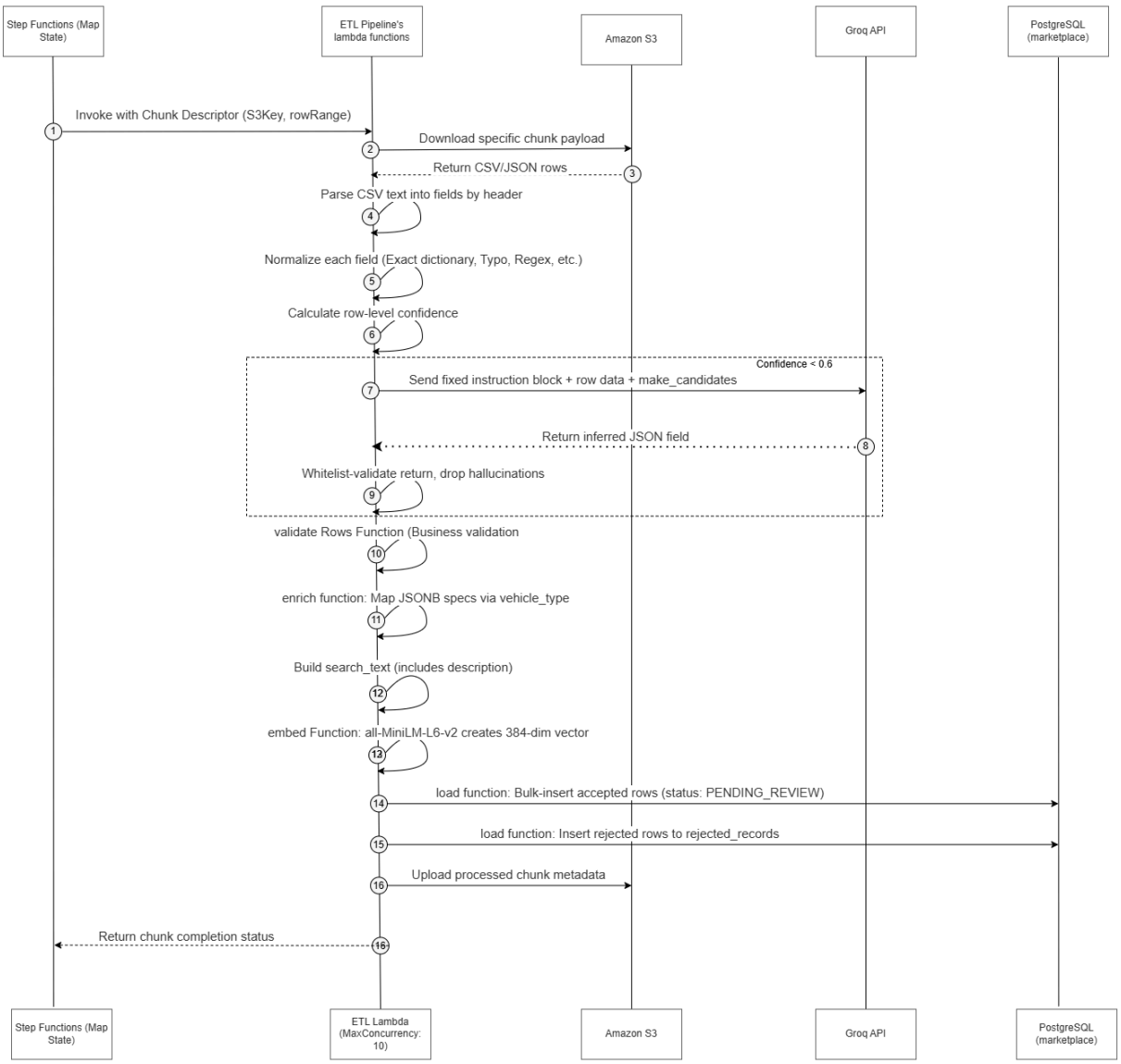


Figure 21: represents the Parallel Data Processing Loop in the AWS Step Functions Map State. Due to limitations set by AWS payload size, the state machine operates on an execution-per-upload basis. The SplitIntoChunks step results in a small array of descriptors pointing to row ranges in S3, which is spread out using the Map state up to 10 concurrent executions in order to keep the database connection pool safe.

Within the Map State, the normalize function parses the normalize function function, and 6-step normalization is performed for each of the fields, while the confidence level for the row is obtained based on the minimum score rule. When the score is less than 0.6, the rows are sent to the Groq API for inference using instruction block and shortlisted makes as candidates. After this, the output is checked through whitelisting, where enrich function produces the JSONB specification for the vehicles based on their type, creating the search_text. embed function creates the vector using all-MiniLM-L6-v2.

6.7 Intelligent Search Execution Sequence

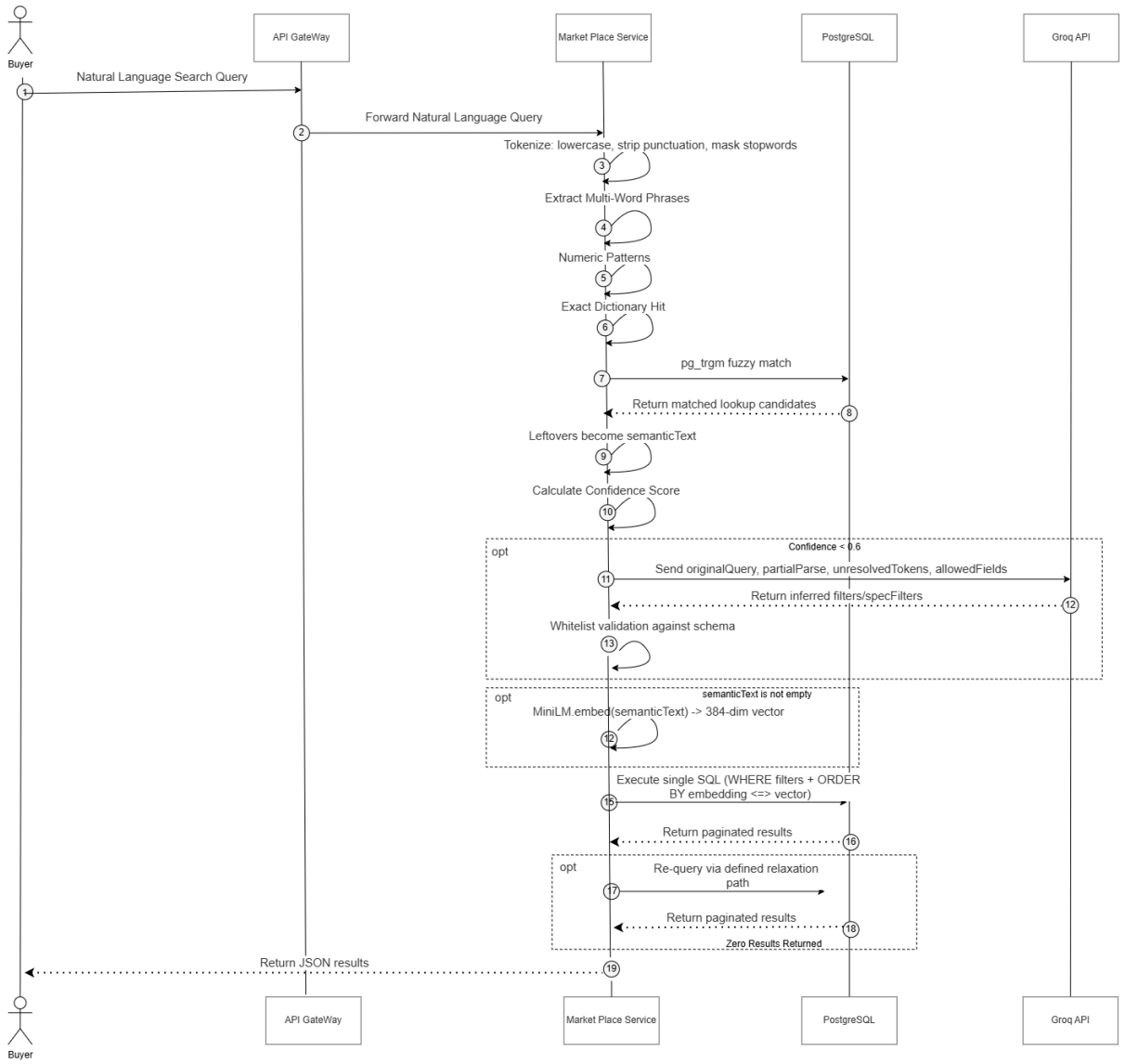


Figure 22: presents the execution flow of a natural-language search query processed by a deterministic 5-stage parser. The tokenizer lowercases text, strips punctuation, splits words, and masks domain-specific stopwords without deleting them. The parser extracts multi-word phrases and numeric patterns, flagging adjacent tokens as digit-adjacent. It then executes exact dictionary hits, strictly resolving Make before Model. Stage 4 performs fuzzy matching using pg_trgm for large vocabularies, utilizing type-gating to reject digit-adjacent tokens outright. Unconsumed tokens are routed into semanticText.

In the case that the confidence score falls below 0.6, the LLM Repair flow kicks in. The LLM fields returned are subject to the Whitelist check no matter what. In case semanticText is not empty, the vector embedding is generated. Finally, the Marketplace service performs a single query using WHERE and ORDER BY clauses.

6.8 Dealer Account Verification State

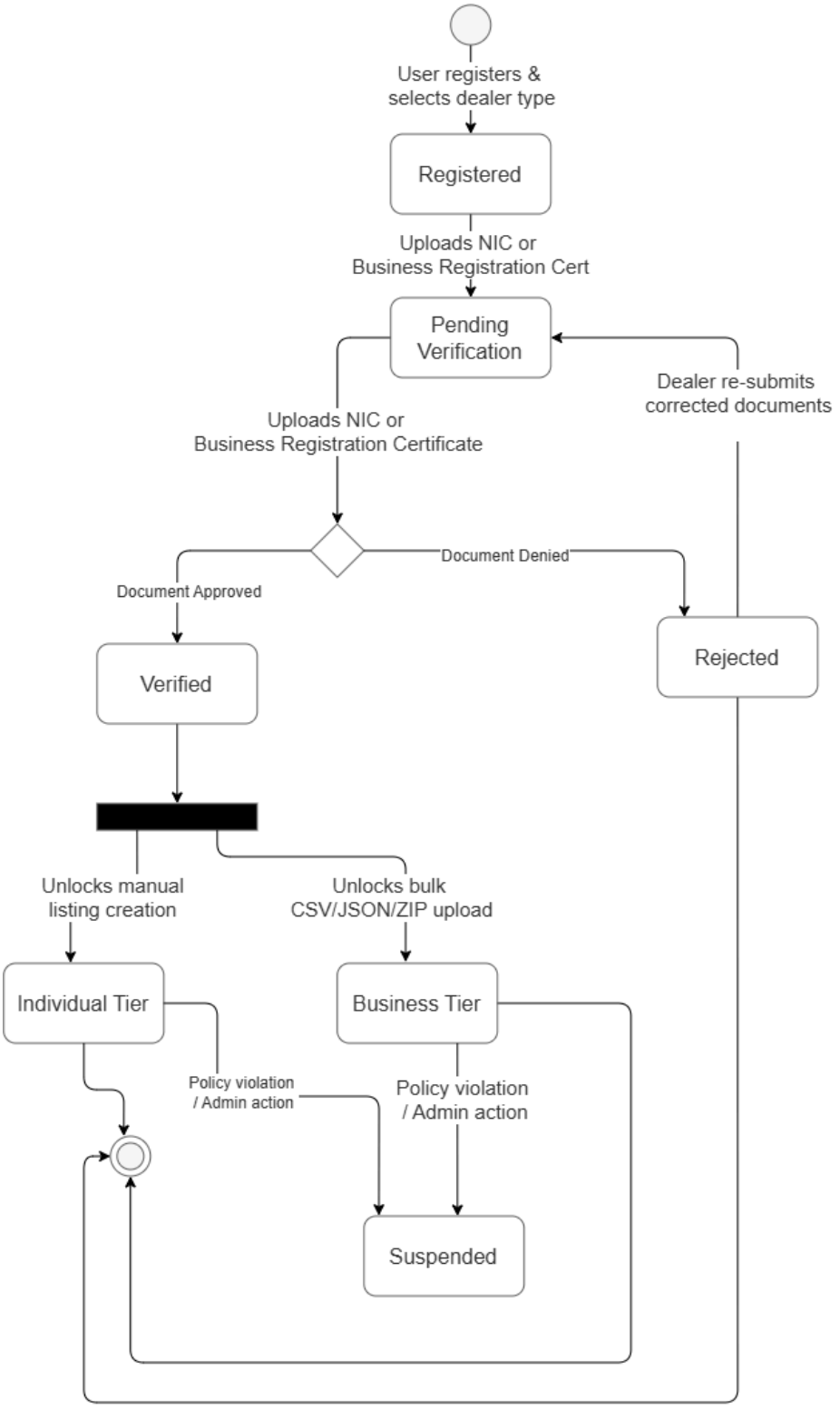


Figure 23: displays the state progression of a Dealer account within the Auth-User Service schema, illustrating the tiered onboarding process. An account begins as REGISTERED and moves to PENDING_VERIFICATION once required documentation is uploaded to S3. An Administrator reviews these documents. Approved documents transition the account to a VERIFIED state, unlocking specific capabilities based on the dealer tier (capped manual listings for Individual, bulk ingestion for

Business). Denied documents transition the state to REJECTED, requiring the dealer to resubmit. Any verified account may be transitioned to SUSPENDED due to policy violations.

6.9 Vehicle Listing State (Row-Level Lifecycle)

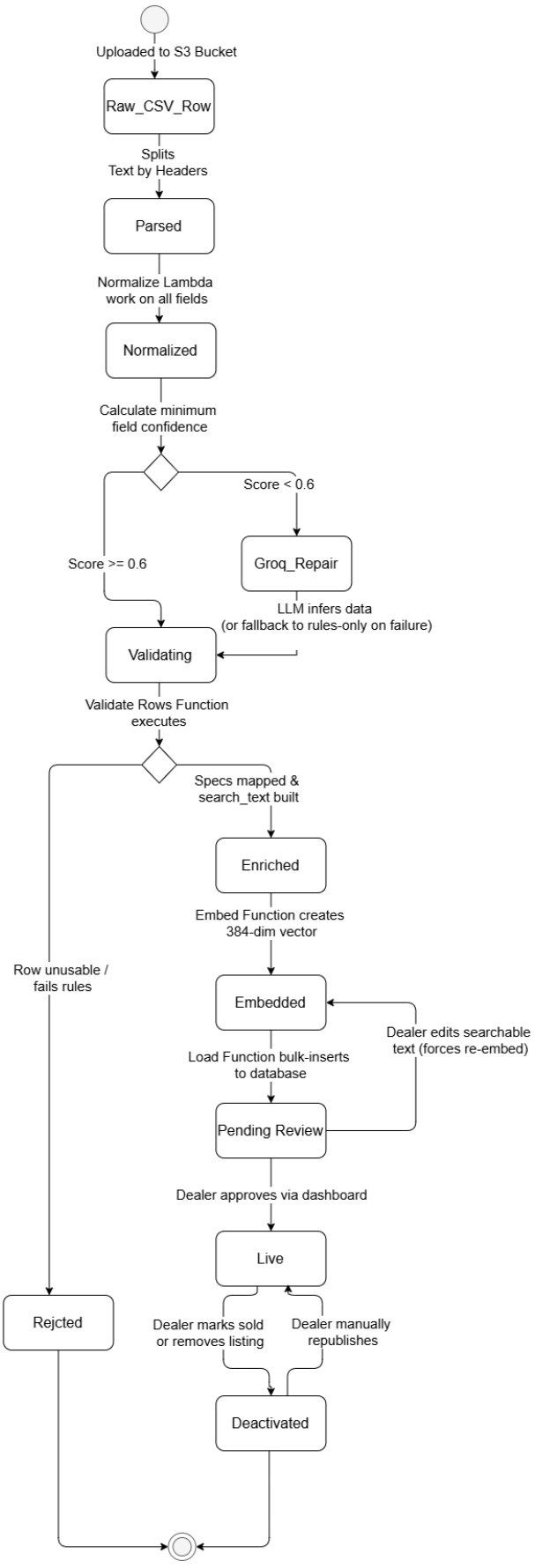

Uploaded to S3 Bucket
Raw_CSV_Row
Splits
Text by Headers
Parsed
Normalize Lambda
work on all fields
Normalized
Calculate minimum
field confidence
Score < 0.6
Score >= 0.6
Groq_Repair
LLM infers data
(or fallback to rules-only on failure)
Validating
Validate Rows Function
executes
Specs mapped &
search_text built
Enriched
Embed Function creates
384-dim vector
Row unusable /
fails rules
Embedded
Dealer edits searchable
text (forces re-embed)
Load Function bulk-inserts
to database
Pending Review
Dealer approves via dashboard
Live
Rejcted
Dealer marks sold
or removes listing
Dealer manually
republishes
Deactivated

Figure 24: outlines the lifecycle of an individual vehicle listing row as it progresses through the ETL pipeline(AWS State Machine) and into the live marketplace. A listing begins as a raw text row in a CSV file and transitions through PARSED, NORMALIZED, and VALIDATED states as it navigates the 6-step normalizer and business rules. Rows falling below a 0.6 confidence threshold temporarily enter a GROQ_REPAIR state.

After enriching the listing with specifications and inserting the vector into it, the Load operation adds the row to the database, thereby moving the listing into a PENDING_REVIEW state. The listing goes into a LIVE state only when the dealer approves the inferred data by the AI through the dashboard. In case the dealer changes the searchable text in a live listing, the listing will move into a PENDING_REVIEW state again.

7. Deployment View

The system deploys entirely on AWS. The physical and logical nodes are organized as follows:

- **Client Device:** Runs a modern web browser and loads the React Single Page Application (SPA).
- **Amazon S3 + CloudFront (optional):** Hosts the built React static assets.
- **Amazon API Gateway (Public):** Serves as the single public entry point, routing north-south HTTPS requests to the correct backend Lambda functions.
- **Internal API Gateway / ALB (Private):** Routes east-west, service-to-service HTTP calls (e.g., Admin's dealer approve/reject call into Auth). It is kept as plain HTTP rather than direct Lambda invocation so local Docker Compose development and AWS deployment share the same code path (ADR-005).
- **AWS Lambda:** Executes all backend code. The 5 logical modules are physically deployed as **18 distinct Lambda functions** to optimize cold starts, memory allocation, and packaging constraints:
    - **Ingestion & ETL Service (13 Lambdas):** 11 dedicated Lambdas orchestrated by Step Functions for the ETL pipeline, plus 2 asynchronous API Lambdas: ingest-upload-api (handling heavy multi-part file streams) and job-status-api (handling high-frequency, lightweight dashboard polling).
    - **Marketplace Service (2 Lambdas):** Asymmetrically split into marketplace-api (handling lightweight CRUD and favourites) and search-api (handling the parser, Groq fallback, MiniLM, and pgvector).
    - **Auth & User Service:** 1 Lambda.
    - **Admin Service:** 1 Lambda.
    - **Notification Service:** 1 Lambda.
- **Lambda Packaging & Dependency Isolation (NFR-27):**
    - *Heavy Inference/Processing:* Any Lambda loading the heavy all-MiniLM-L6-v2 embedding model (embed function and search-api) is deployed as an **Amazon ECR Docker Container Image** sized at 1–3 GB. The images function Lambda utilizing the C++ Sharp library is isolated in a ~2048 MB Docker image.
    - *Lightweight HTTP APIs:* All other HTTP-facing handlers (Auth, Admin, Notification, Marketplace-API, Ingestion-Sync-APIs) and rule-based ETL Lambdas are packaged as **ZIP archives** (or slim Docker images < 50MB). This explicit separation prevents heavy dependencies from increasing cold-start latency on standard, user-facing API calls.

- **AWS Step Functions:** The state machine coordinating one execution per upload job across the 11 ETL Lambdas.
- **Amazon RDS (PostgreSQL + pgvector + pg_trgm):** The single shared relational data store, reachable from Lambda functions inside the deployment VPC. Organized as one schema per service with service-specific database roles.
- **Amazon RDS Proxy:** Connection pooler sitting in front of RDS, required specifically so the ETL Load stage's Map-driven concurrency (MaxConcurrency: 10) does not open 10 raw connections per upload and exhaust the pool shared with live buyer traffic.
- **Amazon S3 (Raw and Staging Buckets):** s3://raw/ holds the original, permanent dealer uploads; s3://staging/ holds per-stage intermediate JSON payloads utilized by Step Functions, configured with a ~7-day expiry lifecycle rule.
- **Amazon SQS:** Durably buffers upload-start messages between the Ingest API and AWS Step Functions.
- **Amazon SES:** Handles outbound transactional emails for the Notification Service.
- **Amazon CloudWatch:** Centralizes logs and metrics for all Lambda functions and the Step Functions execution history.
- **AWS Secrets Manager / SSM Parameter Store:** Securely stores database credentials and third-party API keys (e.g., the Groq API key).
- **AWS IAM:** Enforces least-privilege execution roles scoped strictly per Lambda function.

Deployment diagram

8. Implementation View

8.1 Overview

Source code structure: One repository per deployable unit: auth-user-service, marketplace-service, ingestion-service, admin-service, notification-service, web-frontend, and cloud infrastructure (Terraform). The migrations themselves are kept in one db repository/folder and not in individual services' repositories since the physical RDS database is shared across the team and there are foreign keys that link services (e.g. vehicles.dealer_id -> foreign key to users.id), thus splitting the migrations into different repositories will reintroduce the order dependency despite the coupling being present in the first place. All services will interact with their own tables within the application code.

Inclusion of layers principle: is applicable to all services. Controller and Lambda Handler are skinny (just request processing); Business Logic is implemented in Services, Database access – in Repositories; AWS SDK can be used only at infrastructure/adapters layer. This is the principle that makes possible independent testing of ETL Lambda Handlers without AWS and database.

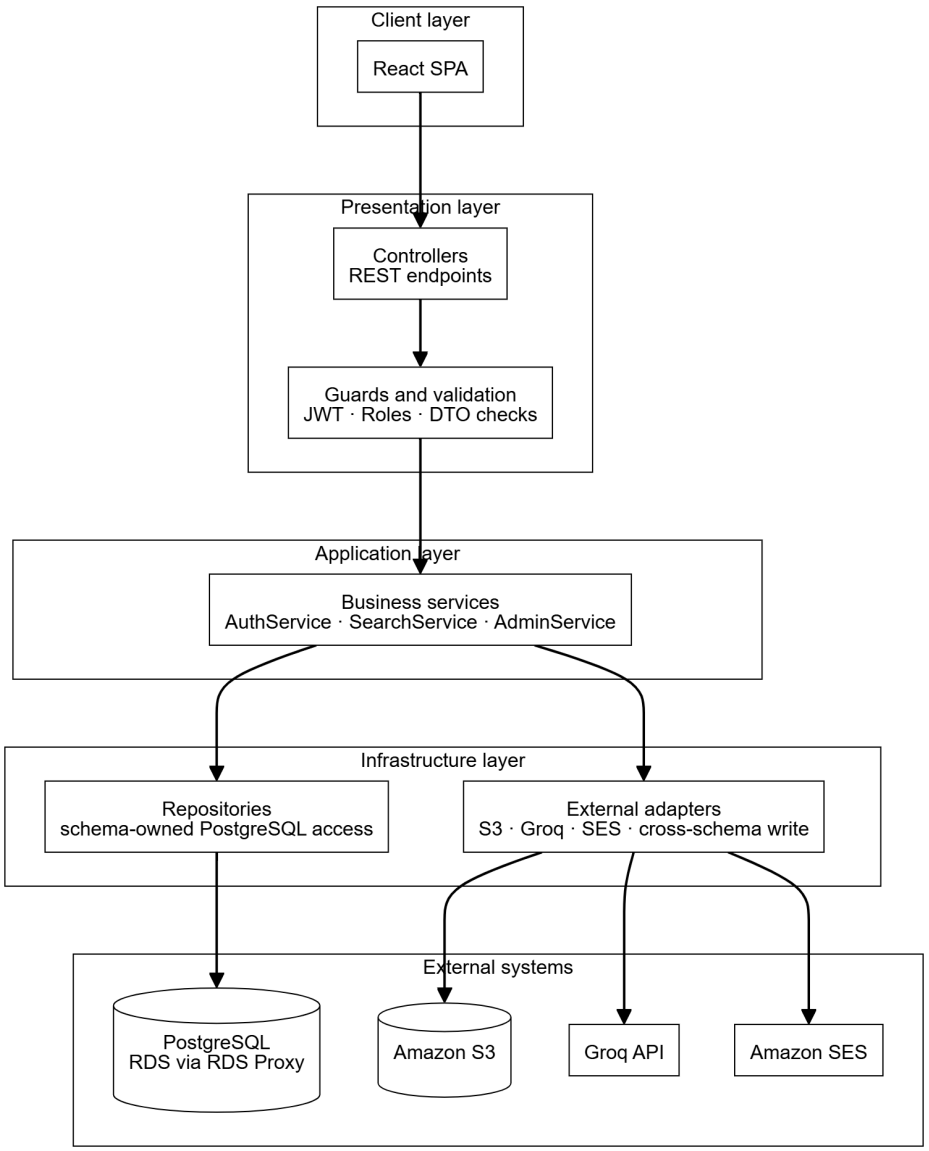


Figure 25: component diagram (Layering)

8.2 Layers

Layer	Contents
Presentation	NestJS controllers (HTTP) and Lambda handler entrypoints (event-triggered); request/response DTO validation only.
Application	Services containing business logic - e.g. SearchService's ranking merge, EnrichHandler's derivation rules, RefreshTokenService's revocation logic.
Persistence	Repositories and ORM entities; the only layer permitted to construct SQL/query-builder calls.

Infrastructure	AWS SDK adapters - S3Adapter, SqsAdapter, SesAdapter, StepFunctionsAdapter, GroqClientAdapter - isolating third-party/AWS API shapes from business logic.
Shared/Common	Guards, decorators, DTO base types, and the shared normalize+embed library consumed by both marketplace-service and ingestion-service.

9. Data View (optional)

The system's persistent data includes a single relational database and two S3 buckets.

Only one Amazon RDS database PostgreSQL is present, which makes use of the following extensions for: Users, DealerProfiles, RefreshTokens, UploadJobs, Vehicles (which has a column vector(384) of embedding, tsvector type search_vector column, and trigram-indexed make/model/description columns), VehicleImages, Favourites, SearchQueries (where the user_id field is optional to track Guest queries), RejectedRecords, ETLStageLogs, Notifications, and AuditLogs. As part of the initial migration, email uniqueness and JSONB attributes of Vehicle type are made sure to ensure that no future migrations are needed.

Iisolated model: although the instance is physically a shared instance, the logical structure of the schemas is such that each service has its own schema, and each service has its own database role. The reads can access other service schemas through explicitly granted narrow SELECT rights – the admin_service_role, for example, has access to marketplace.vehicles and ingestion.upload_jobs for dashboards and ingestion_service_role has access to marketplace.vehicle_dictionaries. Writes stay owned by the schema of the writing service, with one narrow, database-enforced exception: the Ingestion & ETL pipeline's Load stage writes into marketplace.vehicles under a column-scoped grant to ingestion_service_role, confined in code to MarketplaceVehiclesWriteAdapter (section5.2.3, ADR-002). This is the system's only cross-schema write; every other table is written exclusively by its owning service.

Dealer profiles distinguish between dealer types with the help of dealer_type enum (individual, business). Business_registration_no, business_address, and city will be filled if the dealer is of type Business while these fields will remain NULL if the dealer is of type Individual where dealers are distinguished by users.name. Verification documents is a JSONB field consisting of S3 key links to those documents that are actually required (NIC for Individual, business registration certificate for Business along with optional signatory NIC) depending on the specific dealer type, so as to avoid creating partially NULL columns for those documents that are not required for some dealer types, just like in case of vehicles.specs. Verified_by refers to the Administrator who verified or rejected the request. Documents are stored in S3 according to lifecycle-expires policy as dictated by Personal Data Protection Act of 3.11 of SRS.

vehicle_dictionaries is a dictionary table that resides in marketplace-service and consists of the canonical representations of vehicle makes, models, fuels, and transmissions (where models are linked to their parent makes using parent_id field) and a jsonb field named aliases which gets populated over time with misspelling corrections. This table is used to fill in the pg_trgm similarity() comparison logic in both search engine and ETL normalization processes which is a must have due to the fact that similarity() function does not support hard-coded arrays which would lose the index. The ingestion-service takes in this dictionary, caches it in memory during initialization and uses the same

mechanism that Groq uses for their make/model resolution and thus alias promotion is a mere table update.

Use of S3 keys when passing information between Step Functions states and not the row data itself is an immediate result of the 256 KB state payload limit that a single 100-row chunk of embedding vectors exceeds by about 1.8×

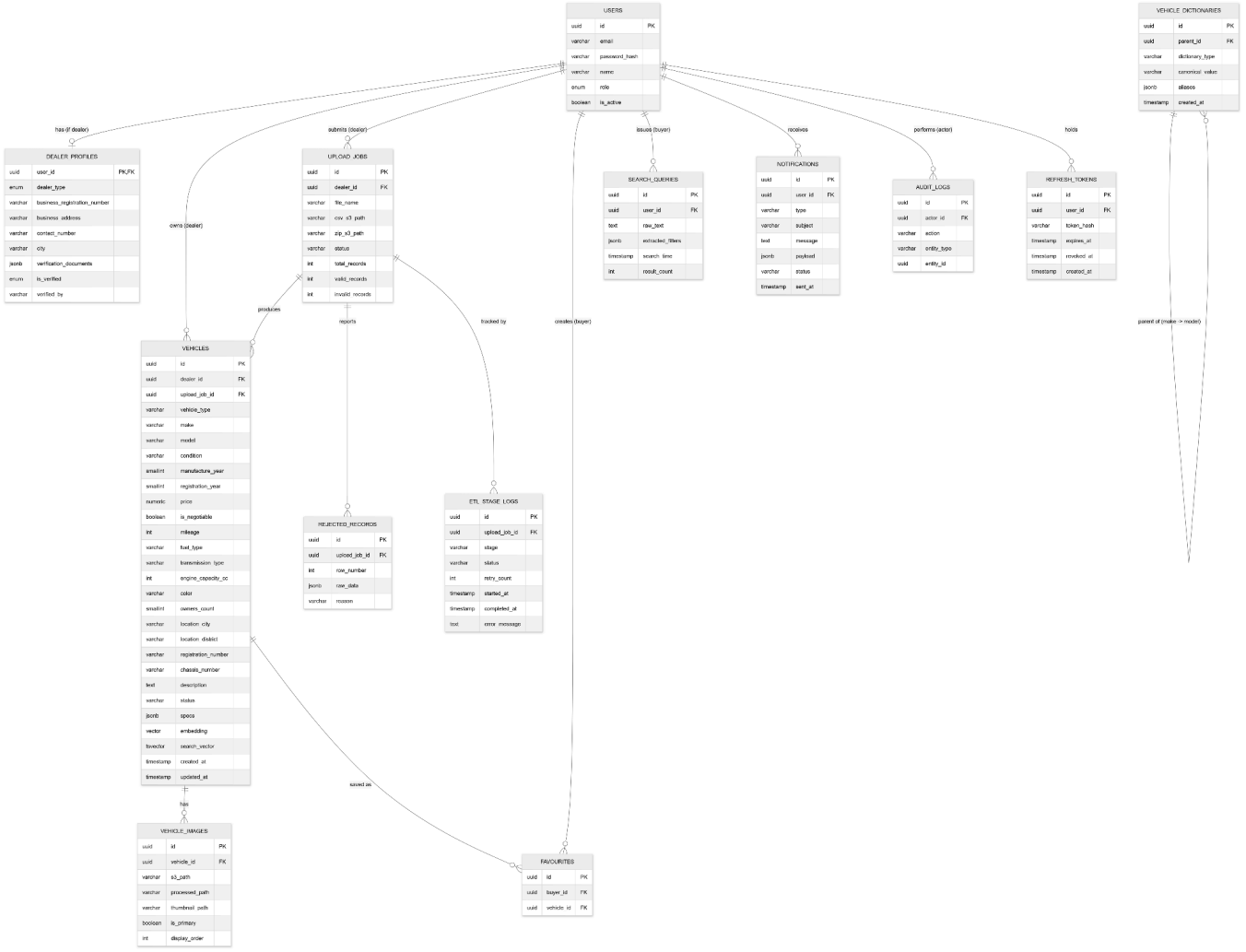


Figure 26: ER diagram of the system

10. Size and Performance

10.1 Size

The chunk size is configured at 100 rows per chunk and not in terms of number of chunks to ensure that the payload size and time of invocation don't vary irrespective of variation in size of the upload; 10,000 row upload becomes 100 chunks and not 200 rows as in the previous case. The Map State

Concurrency is capped at 10 which will ensure that the number of concurrent RDS connections made by the Load Lambda (only stage with live database connection) remains below this number.

Memory Allocation to the Lambdas is done unevenly across 18 lambdas in deployment based on their genuine requirements and not on one uniform basis:

Service Group	Lambda Name	Memory	Runs	Job
Auth & User	auth-api	256-512 MB	per request	Registration, login, profile CRUD
Marketplace	marketplace-api	256-512 MB	per request	Listings CRUD, favourites
Marketplace	search-api	~3008 MB	per query	NL parser, Groq fallback, MiniLM, pgvector
Ingestion	ingest-upload-api	512 MB	per request	Upload intake, multi-part streams, S3/SQS
Ingestion	job-status-api	256 MB	per request	High-frequency job status polling
Admin	admin-api	256-512 MB	per request	Dashboards, user management, reports
Notification	notification-worker	256 MB	per event	SES email dispatch
ETL Pipeline	validateFile function	256 MB	once/upload	Headers, encoding, row count, ZIP integrity
ETL Pipeline	splitChunks function	512 MB	once/upload	Rows -> chunk descriptors -> S3
ETL Pipeline	parseNormalize function	512 MB	per chunk	Parse + dictionaries + regex + confidence
ETL Pipeline	groqNormalize function	256 MB	per flagged	LLM repair (own retry + Catch)
ETL Pipeline	validateRows function	256 MB	per chunk	Mandatory fields -> rejected_records
ETL Pipeline	enrich function	512 MB	per chunk	Derivations, JSONB, search_text

ETL Pipeline	embed function	~3008 MB	per chunk	MiniLM -> 384-dim vector
ETL Pipeline	load function	512 MB	per chunk	Bulk insert -> vehicles as PENDING_REVIEW
ETL Pipeline	images function	~2048 MB	once/upload	Sharp resize + row↔photo matching
ETL Pipeline	aggregate function	256 MB	once/upload	Tally accepted/rejected/flagged
ETL Pipeline	notify function	256 MB	once/upload	Trigger Notification Service

For a representative 1,000-row upload at 100 rows/chunk, this design produces roughly 61 Lambda invocations across one Step Functions execution (versus 10 invocations for a single fused Lambda alternative), a deliberate trade of invocation count for per-stage retry granularity, memory-appropriate billing, and declarative failure handling.

10.2 Performance targets

average API response under 500 ms for CRUD/browse endpoints, under 2 s for natural-language search; design capacity of up to 10,000 listings and 100 concurrent users at prototype scale; graceful fallback to filter/trigram search if semantic ranking degrades.

11. Quality

11.1 Reliability

Step Functions' declarative Retry/Catch on the Groq step means an LLM outage degrades a 1,000-row upload to roughly 95% success (rules-only) instead of failing the entire upload

Row-level idempotent persistence prevents duplicate records on reprocessing: the Load stage's insert is an upsert keyed on (upload_job_id, registration_number), so re-running a chunk after a partial failure cannot create duplicate vehicles rows. If the Load write fails mid-chunk, Step Functions retries the Task state per its declarative Retry/Catch policy; because the upsert is idempotent, retry is safe and produces no duplicate or lost records. Job-intake messages (Ingest API → Step Functions) are separately protected by SQS's dead-letter queue after repeated failures (section3.6.1). this is distinct from the Load stage's direct, non-queued write into marketplace.vehicles.

11.2 Security

JWT-based stateless authentication with revocable, hashed refresh tokens; role-based guards at the controller level

Least-privilege IAM per Lambda function, notably that groqNormalize function resolves make/model against an in-memory table snapshot rather than holding a live database connection, keeping the RDS connection-pool argument intact.

11.3 Scalability

The Map state's per-chunk fan-out and Lambda's automatic scaling mean upload throughput scales with chunk count rather than a fixed worker pool

Fixing rows-per-chunk (not chunk count) keeps this scaling well-behaved as upload size grows.

11.4 Maintainability

The shared normalize+embed library is the single mechanism preventing vocabulary and model drift between the ingestion and search halves of the system

Per-service schema ownership within one physical database keeps logical boundaries clear without the operational cost of five separate RDS instances.

11.5 Extensibility

JSONB per-vehicle-type attributes mean new vehicle types (beyond car/bike/van/truck) require no schema migration

Unmapped CSV columns are preserved as searchable "extras" rather than discarded, so dealer-specific data is never silently lost.

11.6 Observability

Every ETL stage's status, retry count, and error message is logged to etl_stage_logs from the first version

Step Functions provides a live, per-execution graph in the AWS console without any custom tooling

Every provenance-tagged field (source: RULES vs GROQ, with confidence) is retained through to the dealer review UI.

11.7 Requirements Traceability Summary

The following table maps primary SRS requirements to SAD architectural elements. A full matrix is provided in Appendix A.

SRS ID	Requirement summary	SAD element
FR-02, FR-02.1	Tiered dealer registration	ADR-006; Section 4.1.1; AuthService, DealerProfilesRepository
FR-21–FR-24	Hybrid NL search	ADR-004; Section 4.1.4, Section 6.3, Section 6.7; SearchService, DeterministicParser
FR-26–FR-42	Bulk ETL pipeline	ADR-002, ADR-003; Section 4.1.2, Section 4.1.3, Section 6.2, Section 6.5, Section 6.6; Step Functions, 11 ETL Lambdas
FR-42	Pending review gate	Section 9.1 listing status; dealer review UI
FR-44–FR-49	Admin dashboard	Section 4.1.5; AdminService; admin_service_role cross-schema SELECT

FR-50–FR-54	Notifications	notify function -> NotificationService; SesAdapter
NFR-10	RDS Proxy for concurrency	Section 7 Deployment; ADR-003
NFR-12	Lambda memory sizing	Section 10.1 Size table
NFR-26.1	Shared normalize+embed library	ADR-004; Section 5.1 shared library; vehicle_dictionaries table for pg_trgm
Section 3.10	Cross-schema write exception	ADR-002; MarketplaceVehiclesWriteAdapter

Appendix A: Requirements Traceability Matrix (Full)

SRS ID	**Requirement**	**SAD Section / Component**	**Test reference**
FR-01	Register Buyer/Dealer	Section 4 Guest UC; AuthController	Section 3.8.2
FR-02	Dealer type Individual/Business	ADR-006	Section 3.8.2 tier test
FR-03–FR-05	JWT auth + refresh	Section 3.4.1; AuthService, RefreshTokenService	Unit
FR-07	Password hashing	Section 3.4.4	Unit
FR-13.1	Re-embed on searchable edit	Section 9.1 state transition	Integration
FR-14	ETL bulk insert	ADR-002; load function	Section 3.8.2 ADR-002 test
FR-21.2	Groq fallback confidence < 0.6	ADR-004; Section 6.7	Section 3.8.2
FR-22.1	Model parity	ADR-004; shared library	Section 3.8.2 FR-22.1
FR-28	Raw S3 immutable	Section 7 S3 raw bucket	Integration
FR-30.2	S3 keys between states	ADR-003; Section 6.6	Pipeline test
FR-31	202 + Job ID	Section 3.5.1 upload contract	E2E

FR-33.4	Groq for low-confidence rows	ADR-004; groqNormalize function	Pipeline test
FR-35	Image match by registration	images function	Integration
FR-41	Idempotent ETL	Section 3.6.1	Section 3.8.2
FR-42	pending_review default	Section 9.1	Section 3.8.2
FR-55–FR-56	Guest browse/search	Section 4 Guest UC	E2E
NFR-09	API < 500ms; search < 2s	Section 10.2	Performance test
NFR-17	Least-privilege IAM	Section 3.4.3	Infrastructure review
NFR-32	Step Functions orchestration	ADR-003	Pipeline test

Appendix B: Public API Route Catalogue (Illustrative)

Routes below are illustrative; exact paths finalized at implementation. All routes prefixed by API Gateway stage (e.g., /prod).

Auth & User Service

Method	Path	Auth	Description
POST	/auth/register	Public	Register Buyer or Dealer
POST	/auth/login	Public	Issue JWT + refresh token
POST	/auth/refresh	Refresh token	New access token
POST	/auth/logout	JWT	Revoke refresh token
GET	/auth/profile	JWT	View own profile
PATCH	/auth/profile	JWT	Update own profile

Marketplace Service

Method	Path	Auth	Description
GET	/marketplace/vehicles	Public/Guest	Search and filter (FR-19, FR-20)
GET	/marketplace/vehicles/:id	Public/Guest	Vehicle detail
POST	/marketplace/vehicles	Dealer	Create manual listing
PATCH	/marketplace/vehicles/:id	Dealer	Edit own listing
DELETE	/marketplace/vehicles/:id	Dealer	Deactivate (soft)
GET	/marketplace/favourites	Buyer	List favourites
POST	/marketplace/favourites/:vehicleId	Buyer	Add favourite
DELETE	/marketplace/favourites/:vehicleId	Buyer	Remove favourite

Ingestion & ETL Service

Method	Path	Auth	Description
POST	/ingest/upload	Dealer (Business, verified)	Bulk upload; returns 202 + jobId
GET	/jobs/:jobId	Dealer	Poll upload status (FR-32)
GET	/jobs/:jobId/errors	Dealer	Row-level rejection report

Admin Service

Method	Path	Auth	Description
GET	/admin/dashboard	Admin	Aggregate metrics (FR-49)
GET	/admin/users	Admin	List users (FR-44)
POST	/admin/dealers/:id/approve	Admin	Approve dealer -> internal Auth call
POST	/admin/dealers/:id/reject	Admin	Reject dealer
GET	/admin/audit-logs	Admin	Search audit logs (FR-48)
GET	/admin/reports	Admin	Summary reports (FR-46)
GET	/admin/uploads	Admin	All upload jobs (FR-45)

Appendix C: RBAC Permission Matrix

Action	Guest	Buyer	Dealer (Individual)	Dealer (Business)	Admin
Browse/search listings	✓	✓	✓	✓	✓
View vehicle detail	✓	✓	✓	✓	✓
Save favourites	✗	✓	✓*	✓*	✗
Register/login	✓	✓	✓	✓	✗ (seeded)
Manual create listing	✗	✗	✓ (capped)	✓	✗
Bulk CSV/ZIP upload	✗	✗	✗	✓ (verified)	✗
Poll upload job status	✗	✗	✗	✓	✗
Review pending listings	✗	✗	✓ (own)	✓ (own)	✗
Approve/suspend users	✗	✗	✗	✗	✓
View audit logs / reports	✗	✗	✗	✗	✓

Cross-schema dashboard reads	✗	✗	✗	✗	✓ (read-only)

*Dealers may use buyer features if they hold a second account; single role per account (FR-08).